



Python for Data Science - 2305CS303

Lab - 1

Roll No. : 122

Name : Yatharth Sanghavi

1. WAP to print "Hello World".

```
In [27]: print("hello World")
```

hello World

2. WAP to print your address i) using single print ii) using multiple print.

```
In [28]: print("13, Plot No.56, Castle House, Sher-e-punjab Society,mahakali, Andheri")
print("""13,
Plot No.56,
Castle House,
Sher-e-punjab Society,
mahakali,
Andheri""")
```

13, Plot No.56, Castle House, Sher-e-punjab Society,mahakali, Andheri
13,
Plot No.56,
Castle House,
Sher-e-punjab Society,
mahakali,
Andheri

3. WAP to print addition of 2 numbers (without input function).

```
In [1]: a=int(input("Enter a: "))
        b=int(input("enter b: "))
        print(a+b)
```

3

4. WAP to calculate and print average of 2 numbers (without input function).

```
In [2]: a=int(input("Enter a: "))
        b=int(input("enter b: "))
        print((a+b)/2)
```

15.0

5. WAP to add two number entered by user.

```
In [29]: a=int(input("Enter a: "))
         b=int(input("enter b: "))
         print(a+b)
```

30

6. WAP to calculate area of circle.

```
In [6]: a=int(input("enter r: "))
        print(3.14*(a*a))
```

452.16

7. Purposefully raise Indentation Error and Correct it.

```
In [4]: if True:
        print("helo")
        if True:
            print("hello")
```

```
Cell In[4], line 2
    print("helo")
    ^
```

IndentationError: expected an indented block after 'if' statement on line 1

8. WAP to calculate simple interest.

```
In [30]: p=int(input("enter p: "))
         r=int(input("enter r: "))
         t=int(input("enter t: "))
         print((p*r*t)/100)
```

60.0

9. WAP Calculate Area and Circumference of Circle.

```
In [31]: a=int(input("enter r: "))
print(3.14*(a*a))
print(2*3.14*a)
```

```
314.0
62.800000000000004
```

10. WAP to print Multiplication table of given number.

```
In [3]: a=int(input("enter a: "))
for i in range(1,11):
    print(i*a)
```

```
5
10
15
20
25
30
35
40
45
50
```

```
In [ ]: ### 11. WAP to calculate Area of Triangle. (hint: a = h*b*0.5)
```

```
In [5]: b=int(input("enter b: "))
h=int(input("enter h: "))
print(1/2*(b*h))
```

```
100.0
```

12. WAP to convert Degree to Fahrenheit and vice versa.

```
In [32]: b=int(input("enter b: "))
print((b*(9/5))+32)
f=int(input("enter f: "))
print((f-32)*(5/9))
```

```
50.0
-6.666666666666667
```

13.WAP to calculate total marks and Percentage.

```
In [33]: a=int(input("enter a"))
b=int(input("enter b"))
c=int(input("enter c"))
d=int(input("enter d"))
e=int(input("enter e"))
total=a+b+c+d+e
per=(total/100)*5
print(total)
print(per)
```

114
5.699999999999999

14. Calculate Gross Salary

```
In [12]: a=int(input("enter base salary"))
         hdr=(a*10)/100
         da=(a*8)/100
         print(a+hdr+da)
```

118.0

15.Swap Two number

```
In [14]: b=int(input("enter b: "))
         h=int(input("enter h: "))
         temp=b
         b=h
         h=temp
         print(b,h)
```

20 10

16.Swap Two number without using third variable

```
In [15]: b=int(input("enter b: "))
         h=int(input("enter h: "))
         b,h=h,b
         print(b,h)
```

20 10

17. Convert total second into Hours,Minutes and Second

```
In [20]: s=int(input("enter total second"))
         h=int(s/3600)
         m=int((s%3600)/60)
         s=int(s%60)
         print(h,m,s)
```

0 25 0

17. Convert Number of days into Years, Months, days

```
In [26]: d=int(input("enter total days"))
         y=int(d/365)
         m=int((d%365)/30)
         d=int((d%365)%30)
         print(y,m,d)
```

1 0 7



Python for Data Science - 2305CS303

Lab - 2

Roll No. : 122

Name : Yatharth Sanghavi

1. WAP to check whether the given number is Positive or Negative.

```
In [2]: a=int(input("enter a: "))
        if a<0:
            print("number is negative")
        else:
            print("number is positive")
```

number is positive

2. WAP to check whether the given number is Odd or Even.

```
In [3]: a=int(input("enter a: "))
        if a%2==0:
            print("number is even")
        else:
            print("number is odd")
```

number is even

3. WAP to find out Largest number from given two numbers using simple if and ternary operator.

```
In [10]: a=int(input("enter a: "))
         b=int(input("enter b: "))
         if a>b:
             print("A is greater")
         else:
             print("B is greater")
         c = a if a>b else b
         print(c)
```

B is greater
20

4. WAP to find out Largest number from given three numbers.

```
In [14]: a=int(input("enter a: "))
b=int(input("enter b: "))
c=int(input("enter c: "))
if a>b and a>c:
    print("a is greater")
elif b>a and b>c:
    print("b is greater")
else:
    print("c is greater")
```

5. WAP to check whether the given year is Leap year or not.

[If a year can be divisible by 4 but not divisible by 100 then it is leap year but if it is divisible by 400 then it is leap year].

```
In [18]: y=int(input("enter year"))
if (y%4==0 and y%100!=0) or y%400==0:
    print("leap year")
else:
    print("not leap year")
```

6. WAP to display the name of the Day according to the number given by the user.

```
In [22]: day_number = int(input("Enter a day number (1-7): "))

days = ["Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday", "Sunday"]

if 1 <= day_number <= 7:
    print("The day is:", days[day_number - 1])
else:
    print("Invalid input! Please enter a number between 1 and 7.")
```

The day is: Monday

7. WAP to implement simple Calculator which performs (add,sub,mul,div) of two numbers based on user input.

```
In [21]: a=int(input("enter a: "))
b=int(input("enter b: "))
c=int(input("enter 1 for + , 2 for -, 3 for /, 4 for *"))
if c==1:
    print(a+b)
elif c==2:
    print(a-b)
elif c==3:
    print(a/b)
elif c==4:
    print(a*b)
```

```
else:
    print("enter between 1 to 4")
```

70

8. WAP to calculate electricity bill based on following criteria. Which takes the unit from the user.

- First 1 to 50 units – Rs. 2.60/unit
- Next 50 to 100 units – Rs. 3.25/unit
- Next 100 to 200 units – Rs. 5.26/unit
- 200 units – Rs. 8.45/unit

```
In [20]: a=int(input("enter bill: "))
d=0
if a<=50:
    print(a*2.60)
elif a>50 and a<=100:
    print((50*2.60)+((a-50)*3.25))
elif a>100 and a<=200:
    print((50*2.60)+(50*3.25)+((a-100)*5.26))
else:
    print((50*2.60)+(50*3.25)+(50*5.26)+((a-150)*8.45))
```

424.0

9. WAP to simulate movie ticket booking system this program should accept the customer age, day of booking (weekday, weekend) and seat type (silver, gold, platinum) as input.

ticket price should be calculated according to following rule:-

1. seat type (base price:- silver seat:- 150, gold:- 250, platinum:-400)
2. if age below 12 50% discount on base price or age between 12 to 59 no discount or age above 60 discount 30% on base price
3. booking on weekend add additional 20% charge is applied to ticket price after age discount. else no additional charge applied on weekdays

```
In [10]: age = int(input("enter age: "))
day = input("enter day of booking: ")
seattype= input("enter seat type(silver,gold, platinum): ")
price = 150 if seattype == "silver" else 250 if seattype == "gold" else 400 if seattype == "platinum"
p=price
if int(price):
    if age<12:
        dis=int(price*50/100)
        price-=dis
    elif age>=60:
        dis=int(price*30/100)
        price-=dis
    if day=="saturday" or day=="sunday":
```

```
        inc=int(price*20/100)
        price+=inc
    else:
        print("enter proper seat type without typo error")
    print(f"total price : {price} seat type: {seattype} base price: {p} discount: {d}
```

total price : 180 seat type: silver base price: 150 discount: 75



Python for Data Science - 2305CS303

Lab - 3

Roll No. : 122

Name : Yatharth Sanghavi

1. WAP to print 1 to 10.

```
In [1]: for i in range(1,11):  
        print(i)
```

1
2
3
4
5
6
7
8
9
10

2. WAP to print 1 to n.

```
In [3]: n=int(input("enter n: "))  
        for i in range(1,n+1):  
            print(i)
```

1
2
3
4
5
6
7
8
9
10

3.WAP to print odd numbers between 1 to n.

```
In [2]: n=int(input("enter n: "))
        for i in range(1,n+1,2):
            print(i)
```

1
3
5
7
9

4. WAP to print numbers between two given numbers which is divisible by 2 but not divisible by 3.

```
In [4]: n1=int(input("enter n: "))
        n2=int(input("enter n2: "))
        for i in range(n1,n2+1):
            if i%2==0 and i%3!=0:
                print(i)
```

10
14
16
20

5. WAP to print sum of 1 to n numbers.

```
In [8]: n1=int(input("enter n: "))
        sum=0
        for i in range(1,n1+1):
            sum+=i
        print(sum)
```

55

6.WAP to print sum of series 1 + 4 + 9 + 16 + 25 + 36 + ...n.

```
In [10]: n1=int(input("enter n: "))
        sum=0
        for i in range(1,n1+1):
            sum+=i*i
        print(sum)
```

5

7. WAP to print sum of series 1 – 2 + 3 – 4 + 5 – 6 + 7 ... n.

```
In [11]: n1=int(input("enter n: "))
        sum=0
        for i in range(1,n1+1):
            if i%2==0:
                sum-=i
            else:
                sum+=i
        print(sum)
```

4

8. WAP to print multiplication table of given number.

```
In [12]: n1=int(input("enter n: "))
         for i in range(1,11):
             print(f"{n1} * {i}= {n1*i}")
```

```
10 * 1= 10
10 * 2= 20
10 * 3= 30
10 * 4= 40
10 * 5= 50
10 * 6= 60
10 * 7= 70
10 * 8= 80
10 * 9= 90
10 * 10= 100
```

9. WAP to find factorial of the given number.

```
In [13]: n1=int(input("enter n: "))
         sum=1
         for i in range(n1,1,-1):
             sum*=i
         print(sum)
```

```
120
```

10. WAP to find factors of the given number.

```
In [14]: n1=int(input("enter n: "))
         for i in range(1,n1+1):
             if n1%i==0:
                 print(i)
```

```
1
2
5
10
```

11. WAP to find whether the given number is prime or not.

```
In [16]: n1=int(input("enter n: "))
         # flag=True
         for i in range(2,n1):
             if n1%i==0:
                 print("not prime number")
                 break
         else:
             print("prime number")
```

```
prime number
```

12. WAP to print sum of digits of given number.

```
In [18]: n1=int(input("enter n: "))
         sum1=0
```

```
while n1>0:
    digit=n1%10
    sum1+=digit
    n1//=10
print(int(sum1))
```

6

13. WAP to check whether the given number is palindrome or not.

```
In [37]: n1=input("enter n: ")
n2=int(n1[::-1])
n1=int(n1)
if n1==n2:
    print("palindrome")
```

palindrome

14. Fibonacci Number

```
In [64]: n1=int(input("enter n: "))
a, b = 0, 1
for i in range(n1):
    print(a)
    a, b = b, a + b
```

0

1

1

2

3

15. Armstrong Number

```
In [73]: n1=int(input("enter n: "))
l=len(str(n1))
sum=0
t=n1
while n1>0:
    div=n1%10
    sum+= div**l
    n1//=10
if sum == t:
    print("it is Armstrong num")
else:
    print("not armstrong num")
```

it is Armstrong num

16. Perfect number

```
In [77]: n1=int(input("enter n: "))
sum=0
for i in range(1,n1):
    if n1%i==0:
        sum+=i
if sum==n1:
```

```
    print("perfect number")
else:
    print("not perfect number")
```

perfect number

17. GCD

```
In [80]: n1=int(input("enter n: "))
n2=int(input("enter n2: "))
for i in range(1,min(n1,n2)):
    if n1%i==0 and n2%i==0:
        max=i
print(max)
```

4

18. LCM

```
In [81]: n1=int(input("enter n: "))
n2=int(input("enter n2: "))
for i in range(1,min(n1,n2)):
    if n1%i==0 and n2%i==0:
        max=i
print((n1*n2)//max)
```

24

19. count no of digit

```
In [83]: n1=int(input("enter n: "))
print(len(str(n1)))
```

12

20.largest digit

```
In [86]: num = int(input("enter n: "))
digits = list(map(int, str(num)))
digits.sort()
print(digits[-1])
```

3



Python for Data Science - 2305CS303

Lab - 4

Roll No. : 122

Name : Yatharth Sanghavi

1) Character Category Counter

Problem: Count uppercase, lowercase, digits, and special characters in a string.

Sample Inputs & Outputs:

- Input: Python3@Lab → Uppercase: 2, Lowercase: 7, Digits: 1, Special: 1
- Input: HELLO123 → Uppercase: 5, Lowercase: 0, Digits: 3, Special: 0
- Input: hi@#9 → Uppercase: 0, Lowercase: 2, Digits: 1, Special: 2

```
In [6]: i="Python3@Lab"
i2="HELLO123"
i3="HI@#9"
u=0
l=0
d=0
s=0
for ch in i3:
    if ch.isupper():
        u+=1
    elif ch.islower():
        l+=1
    elif ch.isdigit():
        d+=1
    else:
        s+=1
print("count uppercase: ",u," count lowercase: ",l," digits: ",d," special: ",s)

count uppercase:  2  count lowercase:  0  digits:  1  special:  2
```

2) Reverse Each Word

- Input: Python is Easy → nohtyP si ysaE

- Input: Hello World → olleH dlroW
- Input: Learn Python Fast → nraeL nohtyP tsaF

```
In [25]: n=input("enter string: ").split()
for ch in range(len(n)):
    n[ch]=n[ch][-1::-1]
n=" ".join(n)
n
```

Out[25]: 'nohtyp si ysae'

3) Case Pattern Identifier

- PYTHON → Uppercase
- python → Lowercase
- Python Programming → Title Case

```
In [27]: n=input("enter str: ")
if n.isupper():
    print("Uppercase")
elif n.islower():
    print("lowercase")
elif n.istitle():
    print("title case")
```

Uppercase

4) First and Last Occurrence Finder

- programming , g → First:3 Last:10
- banana , a → First:1 Last:5
- hello , z → Not found

```
In [32]: n=input("enter str: ")
n1=input("enter char: ")
f=n.find(n1)
l=n.rfind(n1)
if f== -1 and l== -1:
    print("Not found")
else:
    print(f,l)
```

Not found

5) Word Frequency Counter (ignore the case)

- Python is easy and Python is powerful → python: 2
- Java is popular but java is verbose → java: 2
- C is fast → c:1

```
In [34]: n=input("enter str: ").lower()
n1=input("enter word: ").lower()
count=n.count(n1)
print(count)
```

2

6) Remove Extra Spaces

- Python is fun → Python is fun
- Hello World → Hello World
- NoExtraSpace → NoExtraSpace

```
In [40]: n=input("enter str: ")
n=" ".join(n.split())
n
```

```
Out[40]: 'Python is fun'
```

7) Prefix and Suffix Removal

- unhappy.txt → happy
- pretest.py → test
- unwanted.doc → wanted

```
In [43]: n=input("enter str: ")
pre=input("enter prefix: ")
suf=input("enter suffix: ")
n=n.removeprefix(pre)
n=n.removesuffix(suf)
print(n)
```

```
happy
```

8) Replace Vowels

- Programming → Pr*gr*mm*ng
- Education → *d*c*t**n
- Sky → Sky

```
In [60]: n=input("enter str: ")
n1="aeiou"
n2=""
for ch in n:
    if ch in n1:
        n2+="*"
    else:
        n2+=ch
n2
```

```
Out[60]: 'Pr*gr*mm*ng'
```

9) String Compression

- aaabbccccc → a3b2c4
- xxxyyyz → x3y3z1
- abcd → a1b1c1d1

```
In [6]: n=input("enter str: ")
count=0
l=""
for i in range(1,len(n)):
    if n[i-1]==n[i]:
        count+=1
    else:
        l+=n[i-1]+str(count)
        count=1
if n:
    l += n[-1] + str(count)
l
```

Out[6]: 'a2b3c3'

10) Toggle Case (without using str.swapcase())

- PyTh0n → pYtHoN
- HELlo → heLLo
- Python → pYTHON

```
In [63]: n=input("enter str: ")
n2=""
for ch in n:
    if ch.isupper():
        n2+=ch.lower()
    elif ch.islower():
        n2+=ch.upper()
n2
```

Out[63]: 'pYTHON'

11) Username Validation

valid username rules:

1. only alphabets and digits
2. does not start with a digit
3. length ≥ 6

- User123 → Valid
- 123User → Invalid
- Us@12 → Invalid

```
In [81]: n=input("enter str: ")
if n.isalnum():
    if n[0].isdigit():
        print("Invalid")
    else:
        if len(n)>=6:
            print("Valid")
        else:
            print("Invalid")
```

```
else:
    print("Invalid")
```

Valid

12) Palindrome Checker (ignore the case)

- Madam → Palindrome
- Level → Palindrome
- Python → Not Palindrome

```
In [87]: n=input("enter string: ")
n1=n[::-1]
if n==n1:
    print("Palindrome")
else:
    print("not Palindrome")
```

Palindrome

13) Longest Word Finder

- Python programming is interesting → programming
- I love coding → coding
- Data structures and algorithms → structures

```
In [101... n=input("enter string: ").split()
max=[]
max2=""
for ch in range(len(n)):
    if len(n[ch])>=len(max2):
        max=ch
        max2=n[ch]
n[max]
```

Out[101... 'algorithms'

14) Case Conversion

- python programming
 - Upper: PYTHON PROGRAMMING
 - Lower: python programming
 - Title: Python Programming
 - Capitalize: Python programming

```
In [102... n=input("enter str: ")
print(n.upper())
print(n.lower())
print(n.title())
print(n.capitalize())
```

PYTHON PROGRAMMING
python programming
Python Programming
Python programming

15) Custom Split

- apple,banana,grape
- 10|20|30
- A-B-C-D

In [107...

```
n=input("enter str: ")  
n1=input("enter split char: ")
```



Python for Data Science - 2305CS303

Lab - 5

Roll No. : 122

Name : Yatharth Sanghavi

01) WAP to find sum of all the elements in a List.

```
In [1]: l1=[1,2,3,4,5,6]
        print(sum(l1))
```

21

02) WAP to find largest element in a List.

```
In [2]: l1=[1,2,3,4,5,6]
        print(max(l1))
```

6

03) WAP to find the length of a List.

```
In [3]: l1=[1,2,3,4,5,6]
        print(len(l1))
```

6

04) WAP to interchange first and last elements in a list.

```
In [11]: l1=[1,2,3,4,5,6]
         l1[0],l1[len(l1)-1]=l1[len(l1)-1],l1[0]
         print(l1)
```

[6, 2, 3, 4, 5, 1]

05) WAP to split the List into two parts and append the first part to the end.

```
In [13]: l1=[1,2,3,4,5,6]
m=len(l1)//2
l2=l1[:m]
l3=l1[m:]
l3+=l2
l3
```

Out[13]: [4, 5, 6, 1, 2, 3]

06) WAP to interchange the elements on two positions entered by a user.

```
In [15]: l1=[1,2,3,4,5,6]
n=int(input("enter element index: "))
n1=int(input("enter 2 element index: "))
l1[n],l1[n1]=l1[n1],l1[n]
l1
```

Out[15]: [1, 5, 3, 4, 2, 6]

07) WAP to reverse the list entered by user.

```
In [39]: l1=[]
n=int(input("enter list length: "))
for i in range(n):
    x=input("enter element: ")
    l1.append(x)
l1.reverse()
l1
```

Out[39]: ['8', 'rahul', '5', 'raj', '1']

08) WAP to print even numbers in a list.

```
In [19]: l1=[1,2,3,4,5,6]
for i in range(len(l1)):
    if l1[i]%2==0:
        print(l1[i])
```

2
4
6

09) WAP to count unique items in a list.

```
In [20]: l1=[1,2,3,4,5,6]
s=set(l1)
print(len(s))
```

6

10) WAP to copy a list.

```
In [26]: from copy import deepcopy
l1=[1,2,3,4,5,6]
l2=l1
l3=deepcopy(l1)
print("l2:",l2)
print("l3: ",l3)
```

```
l2: [1, 2, 3, 4, 5, 6]
l3: [1, 2, 3, 4, 5, 6]
```

11) WAP to print all odd numbers in a given range.

```
In [27]: n=int(input("enter first num: "))
n2=int(input("enter second num: "))
for i in range(n,n2+1):
    if i%2!=0:
        print("odd num is: ",i)
```

```
odd num is:  1
odd num is:  3
odd num is:  5
odd num is:  7
odd num is:  9
```

12) WAP to count occurrences of an element in a list.

```
In [28]: l1=[1,2,3,4,5,6,1]
d={}
for i in l1:
    if i not in d:
        d[i]=1
    else:
        d[i]+=1
print(d)
```

```
{1: 2, 2: 1, 3: 1, 4: 1, 5: 1, 6: 1}
```

13) WAP to find second largest number in a list.

```
In [41]: l1=[1,2,3,4,5,6,1]
l1=list(set(l1))
l1.sort()
l1[-2]
```

```
Out[41]: 5
```

14) WAP to extract elements with frequency greater than K.

```
In [29]: k=int(input("enter k: "))
l1=[1,2,3,4,5,6,1]
d={}
for i in l1:
    if i not in d:
        d[i]=1
    else:
        d[i]+=1
```

```
for j,v in d.items():  
    if v==k:  
        print(j)
```

1

15) WAP to create a list of squared numbers from 0 to 9 with and without using List Comprehension.

```
In [31]: s=[i**2 for i in range(0,10)]  
s1=[]  
for i in range(0,10):  
    s1.append(i**2)  
print("s: ",s)  
print("s1: ",s1)
```

```
s: [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]  
s1: [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

16) WAP to create a new list (fruit whose name starts with 'b') from the list of fruits given by user.

```
In [34]: fruit=["apple","banana","grapes","watermelon"]  
fruits=[s for s in fruit if s.lower().startswith('b')]  
fruits
```

```
Out[34]: ['banana']
```

17) WAP to create a list of common elements from given two lists.

```
In [36]: l1=[1,2,3,4,5,6,1]  
l2=[1,2,3]  
s=set(l1) & set(l2)  
list(s)
```

```
Out[36]: [1, 2, 3]
```



Python for Data Science - 2305CS303

Lab - 6

Roll No. : 122

Name : Yatharth Sanghavi

01) WAP to find sum of tuple elements.

Sample Input:

T = (10, 20, 30, 40)

Sample Output:

Sum of tuple elements = 100

```
In [4]: t=(10,20,30,40)
        sum(t)
```

Out[4]: 100

02) WAP to find Maximum and Minimum K elements in a given tuple.

Sample Input:

T = (7, 2, 9, 4, 1, 5)
K = 2

Sample Output:

Minimum 2 elements: (1, 2)
Maximum 2 elements: (9, 7)

```
In [19]: t=(7, 2, 9, 4, 1, 5)
l=list(sorted(t))
print(l)
mi=list(l[:min(l)+1])
ma=list(l[-1:-3:-1])
print("Minimum 2 elements: ",tuple(mi))
print("Maximum 2 elements: ",tuple(ma))
```

```
[1, 2, 4, 5, 7, 9]
Minimum 2 elements: (1, 2)
Maximum 2 elements: (9, 7)
```

03) WAP to find tuples which have all elements divisible by K from a list of tuples.

Sample Input:

```
tuple_list = [(2, 4, 6), (3, 6, 9), (4, 8, 12), (5, 10, 15)]
K = 2
```

Sample Output:

```
Tuples divisible by 2 : [(2, 4, 6), (4, 8, 12)]
```

```
In [20]: tuple_list = [(2, 4, 6), (3, 6, 9), (4, 8, 12), (5, 10, 15)]
K = 2
result=[]
for i in tuple_list:
    a=True
    for j in i:
        if j % 2!=0:
            a=False
            break
    else:
        result.append(i)
result
```

```
Out[20]: [(2, 4, 6), (4, 8, 12)]
```

04) WAP to create a list of tuples from given list having number and its cube in each tuple.

Sample Input:

```
numbers = [1, 2, 3, 4, 5]
```

Sample Output:

```
List of tuples with number and its cube: [(1, 1), (2, 8), (3, 27), (4, 64), (5, 125)]
```

```
In [22]: numbers = [1, 2, 3, 4, 5]
result=[]
for i in numbers:
    j=(i,i**3)
    result.append(j)
result
```

```
Out[22]: [(1, 1), (2, 8), (3, 27), (4, 64), (5, 125)]
```

05) WAP to find tuples with all positive elements from the given list of tuples.

Sample Input:

```
tuple_list = [(1, 2, 3), (-1, 2, 3), (4, 5, 6), (0, 1, 2), (-3, -4, -5)]
```

Sample Output:

```
Tuples with all positive elements: [(1, 2, 3), (4, 5, 6)]
```

```
In [24]: tuple_list = [(1, 2, 3), (-1, 2, 3), (4, 5, 6), (0, 1, 2), (-3, -4, -5)]
result=[]
for i in tuple_list:
    for j in i:
        if j<=0:
            break
    else:
        result.append(i)
result
```

```
Out[24]: [(1, 2, 3), (4, 5, 6)]
```

06) WAP to remove tuples of length K.

Sample Input:

```
tuple_list = [(1, 2), (3, 4, 5), (6,), (7, 8, 9), (10, 11)]
K = 2
```

Sample Output:

```
List after removing tuples of length 2 : [(3, 4, 5), (6,), (7, 8, 9)]
```

```
In [26]: tuple_list = [(1, 2), (3, 4, 5), (6,), (7, 8, 9), (10, 11)]
k = 2
result=[]
for i in tuple_list:
    if len(i)!=k:
```

```
        result.append(i)
result
```

Out[26]: [(3, 4, 5), (6,), (7, 8, 9)]

07) WAP to remove all occurrences of a given element from a tuple.

Sample Input:

```
T = (3, 5, 3, 7, 3, 9)
x = 3
```

Sample Output:

(5, 7, 9)

```
In [27]: T = (3, 5, 3, 7, 3, 9)
x = 3
t=()
for i in T:
    if i!=x:
        t+=(i,)
t
```

Out[27]: (5, 7, 9)

08) WAP to remove duplicates from tuple.

Sample Input:

```
T = (3, 5, 3, 7, 3, 9)
```

Sample Output:

(3, 5, 7, 9)

```
In [29]: T = (3, 5, 3, 7, 3, 9)
s=set(T)
print(tuple(s))
```

(9, 3, 5, 7)

09) WAP to multiply adjacent elements of a tuple and print that resultant tuple.

Sample Input:

```
t = (2, 3, 4, 5)
```

Sample Output:

Resultant tuple after multiplying adjacent elements: (6, 12, 20)

```
In [31]: t = (2, 3, 4, 5)
          T=()
          for i in range(len(t)-1):
              T+=(t[i]*t[i+1],)
          T
```

Out[31]: (6, 12, 20)

10) WAP to test if the given tuple is distinct or not.

Sample Input:

```
tuple1 = (1, 2, 3, 4)
tuple2 = (1, 2, 2, 3)
```

Sample Output:

```
(1, 2, 3, 4) is distinct? -> True
(1, 2, 2, 3) is distinct? -> False
```

```
In [32]: tuple1 = (1, 2, 3, 4)
          tuple2 = (1, 2, 2, 3)
          if len(tuple(set(tuple1)))==len(tuple1):
              print("tuple1 = (1, 2, 3, 4) is distinct? -> True")
          else:
              print("tuple = (1, 2, 3, 4) is distinct? -> False")
          if len(tuple(set(tuple2)))==len(tuple2):
              print("tuple2 = (1, 2, 3, 4) is distinct? -> True")
          else:
              print("tuple2 = (1, 2, 3, 4) is distinct? -> False")
```

```
tuple1 = (1, 2, 3, 4) is distinct? -> True
tuple2 = (1, 2, 3, 4) is distinct? -> False
```

11) WAP to rotate the elements of a tuple to the right by **k** positions.

Sample Input:

```
T = (10, 20, 30, 40, 50)
k = 2
```

Sample Output:

```
(40, 50, 10, 20, 30)
```

```
In [36]: T = (10, 20, 30, 40, 50)
          k = 2
          l=T[k+1:]+T[:k+1]
          l
```

Out[36]: (40, 50, 10, 20, 30)

12) WAP to merge two tuples of equal length alternately.

Sample Input:

```
T1 = (1, 3, 5)
T2 = (2, 4, 6)
```

Sample Output:

```
(1, 2, 3, 4, 5, 6)
```

```
In [39]: T1 = (1, 3, 5)
T2 = (2, 4, 6)
if len(T1)==len(T2):
    T3=(T1+T2)
    print(tuple(sorted(T3)))
else:
    print("Tuple length are not equal")
```

```
(1, 2, 3, 4, 5, 6)
```

13) WAP to create a tuple of squares of elements that are even and greater than 10.

Sample Input:

```
T = (4, 12, 7, 18, 10)
```

Sample Output:

```
(144, 324)
```

```
In [42]: T = (4, 12, 7, 18, 10)
t=[]
for i in T:
    if i %2==0:
        if i>10:
            t.append(i**2)
tuple(t)
```

```
Out[42]: (144, 324)
```

14) WAP to create (index, value) pairs for prime numbers.

Sample Input:

```
T = (4, 7, 9, 11, 15)
```

Sample Output:

```
((1, 7), (3, 11))
```

```
In [63]: def prime(l):
          for j in range(2,l):
              if l%j==0:
                  return False
              else:
                  return True

          T = (4, 7, 9, 11, 15)
          t=()
          for i in range(len(T)):
              b=prime(T[i])
              if b:
                  k=tuple([i,T[i]])
                  t+=(k,)

          t
```

Out[63]: ((1, 7), (3, 11))

15) WAP to split a tuple into even-index and odd-index tuples.

Sample Input:

```
T = ('a', 'b', 'c', 'd', 'e')
```

Sample Input:

```
even = ('a', 'c', 'e')
odd = ('b', 'd')
```

```
In [66]: T = ('a', 'b', 'c', 'd', 'e')
          even=()
          odd=()
          for i in range(len(T)):
              if i%2==0:
                  even+=(T[i],)
              else:
                  odd+=(T[i],)
          print(even)
          print(odd)
```

```
('a', 'c', 'e')
('b', 'd')
```

16) WAP to compute column-wise sum from a tuple of lists.

Sample Input:

```
T = ([1, 2, 3], [4, 5, 6], [7, 8, 9])
```

Sample Output:

```
12 15 18
```

```
In [78]: T = ([1, 2, 3], [4, 5, 6], [7, 8, 9])
result=()
for i in range(len(T)):
    total=0
    for j in range(len(T[i])):
        total+=T[j][i]
    result+=(total,)
print(result)
```

(12, 15, 18)



Python for Data Science - 2305CS303

Lab - 7

Roll No. : 122

Name : Yatharth Sanghavi

01) WAP to find their union, intersection, and difference of the given two sets.

Sample Input:

A = {1, 2, 3, 4}
B = {3, 4, 5, 6}

Sample Output:

Union: {1, 2, 3, 4, 5, 6}
Intersection: {3, 4}
Difference (A-B): {1, 2}

```
In [4]: A = {1, 2, 3, 4}
B = {3, 4, 5, 6}
print("Union:", A|B)
print("Intersection:", A&B)
print("Difference:", A-B)
```

Union: {1, 2, 3, 4, 5, 6}
Intersection: {3, 4}
Difference: {1, 2}

02) WAP to Find elements common to all three given sets.

Sample Input:

A = {1, 2, 3, 4}
B = {2, 3, 5}

C = {2, 3, 6}

Sample Output:

{2, 3}

```
In [5]: A = {1, 2, 3, 4}
        B = {2, 3, 5}
        C = {2, 3, 6}
        print(A&B&C)
```

{2, 3}

03) WAP to remove duplicate elements from a list while preserving unique values.**Sample Input:**

[10, 20, 10, 30, 20, 40]

Sample Output:

{10, 20, 30, 40}

```
In [6]: l=[10, 20, 10, 30, 20, 40]
        s=set(l)
        s
```

Out[6]: {10, 20, 30, 40}

04) WAP to count how many elements are common between two sets.**Sample Input:**

A = {1, 2, 3, 4}
B = {3, 4, 5, 6}

Sample Output:

2

```
In [7]: A = {1, 2, 3, 4}
        B = {3, 4, 5, 6}
        print(len(A&B))
```

2

05) WAP to Merge two dictionaries. If a key exists in both dictionaries, add their values.

Sample Input:

```
Dict1 = {1: 10, 2: 20, 3: 30}
Dict2 = {2: 40, 3: 50, 4: 60}
```

Sample Output:

```
{1: 10, 2: 60, 3: 80, 4: 60}
```

```
In [15]: Dict1 = {1: 10, 2: 20, 3: 30}
Dict2 = {2: 40, 3: 50, 4: 60}
for k,v in Dict2.items():
    if k in Dict1:
        Dict1[k]+=v
    else:
        Dict1[k]=v
Dict1
```

```
Out[15]: {1: 10, 2: 60, 3: 80, 4: 60}
```

06) WAP to Sort a dictionary in ascending order of its values.

Hint: in sorted(), use udf for key parameter

Sample Input:

```
{'Math': 70, 'Physics': 85, 'Chemistry': 60}
```

Sample Output:

```
{'Chemistry': 60, 'Math': 70, 'Physics': 85}
```

```
In [34]: subjects={'Math': 70, 'Physics': 85, 'Chemistry': 60}
print(dict(sorted(subjects.items(),key=(lambda t:t[1]))))
```

```
{'Chemistry': 60, 'Math': 70, 'Physics': 85}
```

07) WAP to find the keys having maximum and minimum values in a dictionary.**Sample Input:**

```
{'A': 50, 'B': 90, 'C': 30}
```

Sample Output:

```
Max Value Key: B
Min Value Key: C
```

```
In [44]: d={'A': 50, 'B': 90, 'C': 30}
print("Max Value key: ",max(d.items(),key=(lambda t:t[1]))[0])
print("Min Value key: ",min(d.items(),key=(lambda t:t[1]))[0])
```

Max Value key: B

Min Value key: C

```
In [81]: d={'A': 50, 'B': 90, 'C': 30}
d[max(d,key=lambda k:d[k])]
```

Out[81]: 90

08) WAP to create a dictionary with numbers from 1 to N as keys and their squares as values.

Sample Input:

N = 5

Sample Output:

{1: 1, 2: 4, 3: 9, 4: 16, 5: 25}

```
In [58]: n=5
d={k+1:v**2 for k,v in enumerate(range(1,n+1))}
d
```

Out[58]: {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}

09) WAP to count the frequency of each character in a string using a dictionary.

Sample Input:

programming

Sample Output:

{'p': 1, 'r': 2, 'o': 1, 'g': 2, 'a': 1, 'm': 2, 'i': 1, 'n': 1}

```
In [62]: s="programming"
d={}
for i in s:
    if i not in d:
        d[i]=1
    else:
        d[i]+=1
d
```

Out[62]: {'p': 1, 'r': 2, 'o': 1, 'g': 2, 'a': 1, 'm': 2, 'i': 1, 'n': 1}

10) WAP to swap keys and values of a dictionary.

Sample Input:

```
{1: 'A', 2: 'B', 3: 'C'}
```

Sample Output:

```
{'A': 1, 'B': 2, 'C': 3}
```

```
In [63]: d={1: 'A', 2: 'B', 3: 'C'}
r={}
for k,v in d.items():
    r[v]=k
r
```

```
Out[63]: {'A': 1, 'B': 2, 'C': 3}
```

11) WAP to find the student with the highest total marks, given student names and their marks.**Sample Input:**

```
{'Amit': [70, 80, 90], 'Riya': [85, 75, 80], 'Neha': [60, 70, 65]}
```

Sample Output:

```
Topper: Amit
```

```
In [69]: d={'Amit': [70, 80, 90], 'Riya': [85, 75, 80], 'Neha': [60, 70, 65]}
print("Topper: ",max(d.items(),key=(lambda t:sum(t[1])))[0])
```

```
Topper: Amit
```

12) WAP to extract keys whose values are even numbers.**Sample Input:**

```
{1: 11, 2: 20, 3: 15, 4: 40}
```

Sample Output:

```
[2, 4]
```

```
In [70]: d={1: 11, 2: 20, 3: 15, 4: 40}
e=[v for k,v in d.items() if v%2==0]
e
```

```
Out[70]: [20, 40]
```

13) WAP to Find common keys between two dictionaries.**Sample Input:**

```
Dict1 = {1: 'A', 2: 'B', 3: 'C'}  
Dict2 = {2: 'X', 3: 'Y', 4: 'Z'}
```

Sample Output:

```
[2, 3]
```

```
In [77]: Dict1 = {1: 'A', 2: 'B', 3: 'C'}  
Dict2 = {2: 'X', 3: 'Y', 4: 'Z'}  
result=[k for k,v in Dict1.items() if k in Dict2]  
result
```

```
Out[77]: [2, 3]
```

14) WAP to handle missing keys in dictionaries.

Sample Input - 1:

```
dict1 = {'a': 5, 'c': 8, 'e': 2}  
key = 'c'
```

Sample Output - 1:

```
8
```

Sample Input - 2:

```
dict1 = {'a': 5, 'c': 8, 'e': 2}  
key = 'd'
```

Sample Output - 2:

```
Key Not Found
```

```
In [84]: dict1 = {'a': 5, 'c': 8, 'e': 2}  
key='c'  
if key in dict1:  
    print(dict1[key])  
else:  
    print("Key Not Found")
```

```
8
```

15) WAP to create a dictionary using dictionary comprehension from two lists containing student names and marks. Include only those students in the dictionary whose marks are greater than or equal to 75.

Sample Input:

```
students = ['Amit', 'Riya', 'Neha', 'Karan']
marks = [78, 72, 85, 60]
```

Sample Output:

```
{'Amit': 78, 'Neha': 85}
```

```
In [91]: students = ['Amit', 'Riya', 'Neha', 'Karan']
marks = [78, 72, 85, 60]
res={k:v for k,v in zip(students,marks) if v >=75}
res
```

```
Out[91]: {'Amit': 78, 'Neha': 85}
```

16) d = {"GFG": [5,7,5,4,5],"is":[6,7,4,3,3],"Best":[9,9,6,5,5]}

OUTPUT

'is'

```
In [106... d = {"GFG": [5,7,5,4,5],"is":[6,7,4,3,3],"Best":[9,9,6,5,5]}
r=[max(d.items(),key=lambda t: len(set(t[1])))[0]]
r
```

```
Out[106... ['is']
```

17) Take string and replace word from dictionary

```
In [119... s="hello world, have a great day"
d={"hello":"hi","world":"earth","great":"wonderful"}
for i in d:
    s=s.replace(i,d[i])
s
```

```
Out[119... 'hi earth, have a wonderful day'
```

18) remove duplicate from string after first occurrence

```
In [123... s="Hello students how are you students how is day going".split()
l=[]

for i in s:
    if i not in l:
        l.append(i)
print(" ".join(l))
```

Hello students how are you is day going



Python for Data Science - 2305CS303

Lab - 8

Roll No. : 122

Name : Yatharth Sanghavi

01) Write a function to calculate BMI given mass and height.
(BMI = mass/h**2)

```
In [13]: def bmi(mass,height):  
         return mass/(height**2)  
  
         mass=100  
         height=2  
         print(bmi(mass,height))
```

25.0

02) Write a function that add first n numbers.

```
In [5]: def addn(n):  
        return sum(range(1,n+1))  
        addn(10)
```

Out[5]: 55

03) Write a function that returns 1 if the given number is Prime or 0 otherwise.

```
In [11]: def prime(n):  
         for i in range(2,n):  
             if n%i==0:  
                 return 0  
         else:  
             return 1  
         r=prime(11)  
         if r==1:  
             print("number is prime")
```

```
else:
    print("number is not prime")
```

number is prime

04) Write a function that returns the list of Prime numbers between given two numbers.

```
In [15]: def prime(n,n2):
          result=[]
          for i in range(n,n2+1):
              for j in range(2,i):
                  if i%j==0:
                      break
              else:
                  result.append(i)
          return result
n=1
n2=10
result=prime(n,n2)
result
```

Out[15]: [1, 2, 3, 5, 7]

05) Write a function that returns True if the given string is Palindrome or False otherwise.

```
In [20]: def palindrome(s):
          return s == s[::-1]
r=palindrome("nayan")
r
```

Out[20]: True

06) Write a function that returns the sum of all the elements of the list.

```
In [21]: def suml(l):
          return sum(l)
l=[1,2,3,4,5]
r=suml(l)
r
```

Out[21]: 15

07) Write a function to calculate the sum of the first element of each tuples inside the list.

```
In [33]: l=[(1,2,3),(1,2,3),(1,2,3)]
          r=0
          for i in range(len(l)):
              r+=l[i][0]
          r
```

Out[33]: 3

08) Write a recursive function to find nth term of Fibonacci Series.

```
In [99]: def fibonacci(n):
          l=[0,1]
          for i in range(n):
              p=l[-1]
              pn=l[-2]
              r=p+pn
              l.append(r)
          return l[n]
          n=5
          print(fibonacci(n))
```

5

09) Write a function to get the name of the student based on the given rollno.

Example: Given dict1 = {101:'Ajay', 102:'Rahul', 103:'Jay', 104:'Pooja'} find name of student whose rollno = 103

```
In [48]: dict1 = {101:'Ajay', 102:'Rahul', 103:'Jay', 104:'Pooja'}
          rollno = 103
          print(dict1.get(rollno))
```

Jay

10) Write a function to get the sum of the scores ending with zero.

Example : scores = [200, 456, 300, 100, 234, 678]

Ans = 200 + 300 + 100 = 600

```
In [60]: def sum_endzero(scores):
          r=0
          for i in scores:
              if i%10==0:
                  r+=i
          return r
          scores = [200, 456, 300, 100, 234, 678]
          print(sum_endzero(scores))
```

600

11) Write a function to invert a given Dictionary.

hint: keys to values & values to keys

Before : {'a': 10, 'b':20, 'c':30, 'd':40}

After : {10:'a', 20:'b', 30:'c', 40:'d'}

```
In [62]: d= {'a': 10, 'b':20, 'c':30, 'd':40}
r={v:k for k,v in d.items()}
r
```

```
Out[62]: {10: 'a', 20: 'b', 30: 'c', 40: 'd'}
```

12) Write a function that returns the number of uppercase and lowercase letters in the given string.

example : Input : s1 = AbcDEfgh ,Oupput : no_upper = 3, no_lower = 5

```
In [66]: s1 = "AbcDEfgh"
no_upper=0
no_lower=0
for i in s1:
    if i.isupper():
        no_upper+=1
    elif i.islower():
        no_lower+=1
print("no_upper=",no_upper)
print("no_lower=",no_lower)
```

```
no_upper= 3
```

```
no_lower= 5
```

13) Write a lambda function to get smallest number from the given two numbers.

```
In [72]: n=10
n2=5
r=(lambda n,n2:n if n<n2 else n2)(n,n2)
print(r)
```

```
5
```

14) For the given list of names of students, extract the names having more that 7 characters. Use filter().

```
In [87]: student=['rahul','ram','ravan','krish','sachin','ramanandan']
r= list(filter(lambda s:len(s)>=7,student))
r
```

```
Out[87]: ['ramanandan']
```

15) For the given list of names of students, convert the first letter of all the names into uppercase. use map().

```
In [90]: student=['rahul','ram','ravan','krish','sachin','ramanandan']
r=list(map(lambda s:s.title(),student))
r
```

```
Out[90]: ['Rahul', 'Ram', 'Ravan', 'Krish', 'Sachin', 'Ramanandan']
```

16) Write udfs to call the functions with following types of arguments:

1. Positional Arguments
2. Keyword Arguments
3. Default Arguments
4. Variable Length Positional(*args) & variable length Keyword Arguments (**kwargs)
5. Keyword-Only & Positional Only Arguments

In [117...

```
def Positional(a,b,c):
    print(a,b,c)
def Keyword(a,b,c):
    print(a,b,c)
def Default(c):
    print(c)
def VariableLP(*c):
    print(c)
def VariableLK(**k):
    print(k)
def keywordo(*,a,b,c):
    print(a,b,c)
def positionalo(a,b,c,/):
    print(a,b,c)

Positional(1,2,3)
Keyword(2,b=3,c=1)
Default(4)
VariableLP(1,2,3,4)
VariableLK(a=10,b=20,c=20)
keywordo(a=0,b=0,c=0)
positionalo(1,2,3)
```

```
1 2 3
2 3 1
4
(1, 2, 3, 4)
{'a': 10, 'b': 20, 'c': 20}
0 0 0
1 2 3
```



Python for Data Science - 2305CS303

Lab - 9

Roll No. : 122

Name : Yatharth Sanghavi

```
!pip install numpy
```

Requirement already satisfied: numpy in /usr/local/lib/python3.13/dist-packages (2.1.3)

1. Create a NumPy array of 10 student marks and display its `dtype`, `size`, `ndim`, `shape`, `itemsize`, and `nbytes`.

```
import numpy as np
student=np.array([10,20,30,40,50,60,70,80,90,100])
print(student.dtype)
print(student.size)
print(student.ndim)
print(student.shape)
print(student.itemsize)
print(student.nbytes)
```

```
int64
10
1
(10,)
8
80
```

2. Create NumPy arrays using zeros, ones, full, arange, linspace, and identity functions.

```
zero=np.zeros(4)
ones=np.ones(4)
full=np.full((2,3),4)
arange=np.arange(1,10)
linspace=np.linspace(1,2,4)
identity=np.identity(3)
print("Zero:",zero)
print("Ones",ones)
print("Full:",full)
print("Arange:",arange)
print("Linspace:",linspace)
print("Identity:",identity)
```

```
Zero: [0. 0. 0. 0.]
Ones [1. 1. 1. 1.]
Full: [[4 4 4]
 [4 4 4]]
Arange: [1 2 3 4 5 6 7 8 9]
Linspace: [1.          1.33333333 1.66666667 2.         ]
Identity: [[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
```

3. Create an array of numbers from 1 to 25, reshape it into a 5×5 matrix, and display its transpose.

```
r=np.array(range(1,26))
print(r.reshape(5,5))
```

```
[[ 1  2  3  4  5]
 [ 6  7  8  9 10]
 [11 12 13 14 15]
 [16 17 18 19 20]
 [21 22 23 24 25]]
```

4. Generate marks of 20 students randomly between 0 and 100 and find their highest, lowest, and average marks.

```
marks=np.random.randint(0,100,20)
print("Highest",marks.max())
print("Lowest",marks.min())
print("Average:",np.average(marks))
```

```
Highest 98
Lowest 4
Average: 43.6
```

5. Generate random numbers using `rand()`, `randn()`, `uniform()`, and `normal()`. Use a seed to reproduce the results.

```
np.random.seed(1)
rand=np.random.rand()
randn=np.random.randn()
uniform=np.random.uniform()
normal=np.random.normal()
print(rand)
print(randn)
print(uniform)
```

```
0.4191945144032948
-1.0170141364670207
0.8781174363909454
```

6. Given roll numbers 101–120, randomly select 5 students without replacement and demonstrate shuffling and permutation.

```
student=np.array(range(101,120))
print(np.random.choice(student,5))
np.random.shuffle(student)
print("shuffling:",student)
print("Permutation: ",np.random.permutation(student))
```

```
[102 103 113 108 103]
shuffling: [109 104 105 117 113 102 106 119 114 118 112 108 116 115 107 103 110 101
111]
Permutation: [105 106 107 101 117 110 116 108 103 118 114 119 109 112 113 111 104 115
102]
```

7. Given marks of 15 students, calculate sum, mean, median, minimum, maximum, variance, and standard deviation.

```
student=np.array(np.random.randint(20,50,15))
print("sum",student.sum())
print("mean",student.mean())
print("median",np.median(student))
print("minimum",student.min())
print("maximum",student.max())
print("Variance",student.var())
print("SD: ",student.std())
```

```
sum 507
mean 33.8
median 34.0
minimum 20
maximum 49
```

```
Variance 79.89333333333333
SD: 8.938307073116997
```

8. Given daily sales for 10 days, calculate total sales, average sales, cumulative sales, and cumulative product.

```
student=np.array(np.random.randint(100,200,10))
print("Total sales:",student.sum())
print("Avg sales: ",np.average(student))
print("Cumulative Sales: ",student.cumsum())
print("Cumulative Product:",student.cumprod())
```

```
Total sales: 1497
Avg sales: 149.7
Cumulative Sales: [ 177  327  506  647  811  994 1118 1238 1382 1497]
Cumulative Product: [          177          26550          4752450
                    670095450      109895653800      20110904645400
                    2493752176029600      2992502611233552000      6198549454372384768
                    -6589831621848264704]
```

9. Create two NumPy arrays and perform addition, subtraction, multiplication, division, floor division, modulus, and exponentiation.

```
num1=np.array(np.random.randint(10,20,5))
num2=np.array(np.random.randint(30,40,5))
print("num1",num1,"num2",num2)
print(num1+num2)
print(num2-num1)
print(num1*num2)
print(num2/num1)
print(num2//num1)
print(num1%num2)
print(num1^num2)
```

```
num1 [10 16 12 14 13] num2 [36 37 36 33 30]
[46 53 48 47 43]
[26 21 24 19 17]
[360 592 432 462 390]
[3.6      2.3125      3.      2.35714286  2.30769231]
[3 2 3 2 2]
[10 16 12 14 13]
[46 53 40 47 19]
```

10. Create an unsorted array of student marks and display it in ascending and descending order. Also find the sorted indices.

```
num1=np.array(np.random.randint(10,20,5))
print(num1)
num1.sort()
print("Ascending", num1)
print("Descending",num1[::-1])
num1.argsort()
```

```
[12 16 16 10 11]
Ascending [10 11 12 16 16]
Descending [16 16 12 11 10]
array([0, 1, 2, 3, 4])
```

11. Given the following NumPy array:

Write a program to display:

- First 5 elements
- Last 4 elements
- Elements from index 2 to 6
- Every alternate element

```
arr = np.array([10, 20, 30, 40, 50, 60, 70, 80, 90, 100])
print("First five:",arr[:5])
print("Last Four:",arr[-4:])
print("2 to 6 index",arr[2:7])
print("alternate",arr[::2])
```

```
First five: [10 20 30 40 50]
Last Four: [ 70  80  90 100]
2 to 6 index [30 40 50 60 70]
alternate [10 30 50 70 90]
```

12. Given the following NumPy array:

Write a program to display:

- The array in reverse order
- Every second element in reverse order
- Last 5 elements in reverse order

```
arr = np.array([5, 10, 15, 20, 25, 30, 35, 40, 45, 50])
print("Reverse:", arr[::-1])
print("every second element", arr[::-2])
print("last five:", arr[-5:-1])
```

```
Reverse: [50 45 40 35 30 25 20 15 10  5]
every second element [50 40 30 20 10]
last five: [50 45 40 35]
```

13. Given following NumPy Array:

Write a program to display:

- First two rows
- Last two rows
- Alternate rows
- Rows from index 1 to 3

```
arr = np.array([
    [10, 20, 30, 40],
    [50, 60, 70, 80],
    [90, 100, 110, 120],
    [130, 140, 150, 160]
])
print(arr[0:2])
print(arr[:-3:-1])
print(arr[:2])
print(arr[0:3])
```

```
[[10 20 30 40]
 [50 60 70 80]]
[[130 140 150 160]
 [ 90 100 110 120]]
[[ 10  20  30  40]
 [ 90 100 110 120]]
[[ 10  20  30  40]
 [ 50  60  70  80]
 [ 90 100 110 120]]
```

14. Given following NumPy Array:

Write a program to display:

- First two columns
- Last three columns
- First three rows and last two columns
- A 2x3 sub-array from the center

```
arr = np.array([
    [1, 2, 3, 4, 5],
    [6, 7, 8, 9, 10],
    [11, 12, 13, 14, 15],
    [16, 17, 18, 19, 20]
])
print(arr[:, :2])
print(arr[:, -3::])
print(arr[3, -2::])
```

```
[[ 1  2]
 [ 6  7]
 [11 12]
 [16 17]]
```

```
[[ 3  4  5]
 [ 8  9 10]
 [13 14 15]
 [18 19 20]]
[[ 4  5]
 [ 9 10]
 [14 15]]
```

15. Given following NumPy Array:

Write a program to display:

- The matrix with rows in reverse order
- The matrix with columns in reverse order
- The matrix with both rows and columns reversed
- Alternate rows and alternate columns

```
arr = np.array([
    [1, 2, 3, 4],
    [5, 6, 7, 8],
    [9, 10, 11, 12],
    [13, 14, 15, 16]
])
print(arr[::-1])
print(arr[:,::-1])
print(arr[::-1,::-1])
print(arr[::-2,::2])
```

```
[[13 14 15 16]
 [ 9 10 11 12]
 [ 5  6  7  8]
 [ 1  2  3  4]]
[[ 4  3  2  1]
 [ 8  7  6  5]
 [12 11 10  9]
 [16 15 14 13]]
[[16 15 14 13]
 [12 11 10  9]
 [ 8  7  6  5]
 [ 4  3  2  1]]
[[ 1  3]
 [ 9 11]]
```

15. Given an array of marks, write a program to display students who scored more than 75 marks.

```
marks = np.array([45, 78, 62, 89, 35, 91, 56, 73, 40, 84])
print(marks[marks>75])
```

```
[78 89 91 84]
```

16. Given an array, write a program to display elements that are greater than 30 and less than 70.

```
arr = np.array([12, 25, 37, 44, 51, 63, 72, 81, 90, 18])
print(arr[arr>30] and arr[::2])
```

```
[37 44 51 63 72 81 90]
```

17. Given an array, write a program to display elements that are divisible by 5 and greater than 25, and then display elements that are either less than 20 or greater than 45.

```
arr = np.array([10, 15, 20, 25, 30, 35, 40, 45, 50, 55])
print(arr[(arr % 5 == 0) & (arr > 25)])
print(arr[(arr < 20) | (arr > 45)])
```

```
[30 35 40 45 50 55]
[10 15 50 55]
```




Python for Data Science - 2305CS303

Lab - 10

Roll No. : 122

Name : Yatharth Sanghavi

```
import pandas as pd
import numpy as np
```

1. Series in Pandas

Create a Pandas Series containing the marks of 8 students. Display the Series, its index, values, and data type.

```
s= pd.Series([30,40,50,34,45,56,67,45,34,87])
print(s)
```

```
0    30
1    40
2    50
3    34
4    45
5    56
6    67
7    45
8    34
9    87
dtype: int64
```

2. Creating Series from Different Data

Create three Pandas Series: i) from a Python list, ii) from a NumPy array, and iii) from a dictionary. Display all three Series.

```
s= pd.Series([30,40,50,34,45,56,67,45,34,87])
s1=pd.Series(np.array([10,20,30]))
d={'a':10,'b':20}
s2=pd.Series(d)
print("-----list-----")
print(s)
print("-----numpy array-----")
print(s1)
print("-----dictionary-----")
print(s2)
```

```
-----list-----
0    30
1    40
2    50
3    34
4    45
5    56
6    67
7    45
8    34
9    87
```

```
dtype: int64
-----numpy array-----
0    10
1    20
2    30
dtype: int64
-----dictionary-----
a    10
b    20
dtype: int64
```

3. Custom Index in Series

Create a Pandas Series containing the marks of 5 students with student names as the index. Access the marks of two specific students using their labels.

```
stu={'ram':20, 'ravan':30, 'rrr':40, 'ddd':50, 'rt':60}
marks=pd.Series(stu)
print(marks['ram'])
print(marks['ravan'])
```

```
20
30
```

4. Creating Time Series

Create a date range for 10 consecutive days starting from '2026-09-01' and create a Pandas Series containing daily sales values for those dates. Display the resulting time series.

```
dates=pd.date_range(start='2026-09-01', periods=9)
sales=pd.Series([40,50,60,60,20,60,56,12,45], index=dates)
print(sales)
```

```
2026-09-01    40
2026-09-02    50
2026-09-03    60
2026-09-04    60
2026-09-05    20
2026-09-06    60
2026-09-07    56
2026-09-08    12
2026-09-09    45
Freq: D, dtype: int64
```

5. Time Series with Frequency

Create a time series for the first 6 months of 2026 using a suitable date frequency. Store monthly revenue values in the Series and display the result.

```
dates=pd.date_range(start='2026', periods=6, freq='ME')
sales=pd.Series([40,50,60,60,20,60], index=dates)
print(sales)
```

```
2026-01-31    40
2026-02-28    50
2026-03-31    60
2026-04-30    60
2026-05-31    20
2026-06-30    60
Freq: ME, dtype: int64
```

6. Creating a DataFrame

Create a DataFrame named `students` containing the following columns: RollNo, Name, Department, Marks, and City. Add records for at least 6 students. Display the DataFrame.

```
d={'rollno':[101,102,103,104,105,106],
  'Name':['raj','rahul','ram','ravan','rohan','ritik'],
  'Department':['CE','CE','MANAGEMENT','MANAGEMENT','CE','MANAGEMENT'],
  'Marks':[30,40,50,60,70,80],
  'City':['rajkot','raipur','raigadh','rampur','rome','ranchi']}
student=pd.DataFrame(d)
print(student)
```

	rollno	Name	Department	Marks	City
0	101	raj	CE	30	rajkot
1	102	rahul	CE	40	raipur
2	103	ram	MANAGEMENT	50	raigadh
3	104	ravan	MANAGEMENT	60	rampur
4	105	rohan	CE	70	rome
5	106	ritik	MANAGEMENT	80	ranchi

7. DataFrame from Dictionary

Create a DataFrame from a dictionary containing information about 5 products: ProductID, ProductName, Category, Price, and Quantity. Display the DataFrame and its columns.

```
p={'ProductID':[101,102,103,104,105,106],
  'ProductName':['raj','rahul','ram','ravan','rohan','ritik'],
  'Category':['CE','CE','MANAGEMENT','MANAGEMENT','CE','MANAGEMENT'],
  'Price':[300,400,500,600,700,800],
  'Quantity':[10,20,201,305,604,802]}
product=pd.DataFrame(p)
print(product)
```

	ProductID	ProductName	Category	Price	Quantity
0	101	raj	CE	300	10
1	102	rahul	CE	400	20
2	103	ram	MANAGEMENT	500	201
3	104	ravan	MANAGEMENT	600	305
4	105	rohan	CE	700	604
5	106	ritik	MANAGEMENT	800	802

8. Conditional Selection

Using the `students` DataFrame, display: i) students having Marks greater than 70, ii) students belonging to the 'CSE' department, and iii) students having Marks between 60 and 80.

```
print("Marks greater than 70:-\n",student[student['Marks']>70])
print("Belong to CE:-\n",student[student['Department']=='CE'])
print("Marks between 60 and 80:-\n",student[(student['Marks']>60) & (student['Marks']<80)])
```

```
Marks greater than 70:-
  rollno  Name  Department  Marks  City
5    106  ritik  MANAGEMENT    80  ranchi
Belong to CE:-
  rollno  Name  Department  Marks  City
0    101   raj           CE     30  rajkot
1    102  rahul           CE     40  raipur
4    105  rohan           CE     70   rome
Marks between 60 and 80:-
  rollno  Name  Department  Marks  City
4    105  rohan           CE     70   rome
```

9. Multiple Conditions

Using the `students` DataFrame, select students who belong to the 'CSE' department and have Marks greater than or equal to 70. Also display students who belong to either 'CSE' or 'IT'.

```
print("Marks between 60 and 80:-\n",student[(student['Department']=='CE') & (student['Marks']>=70)])
```

```
Marks between 60 and 80:-
  rollno  Name  Department  Marks  City
4    105  rohan           CE     70   rome
```

10. Conditional Selection on DataFrame Columns

From the `products` DataFrame, display products whose Price is greater than 500 and Quantity is less than 20. Then display only the ProductName and Price columns of the selected records.

```
print(product[(product['Quantity']<20) & (product['Price']>=200)][['ProductName','Price']])
```

	ProductName	Price
0	raj	300

11. Read CSV File

Create a CSV file named `students.csv` containing student records. Read the CSV file using Pandas and display the complete DataFrame. Also display its first 5 records.

```
url = 'https://raw.githubusercontent.com/ShapeLab/ZooidsCompositePhysicalizations/master/Zooid_Vis/bin/data/student-dataset'
student = pd.read_csv(url)
print(student.head())
```

id	name	nationality	city
0	Kiana Lor	China	Suzhou
1	Joshua Lonaker	United States of America	Santa Clarita
2	Dakota Blanco	United States of America	Oakland
3	Natasha Yarusso	United States of America	Castro Valley
4	Brooke Cazares	Brazil	São José dos Campos

	latitude	longitude	gender	ethnic.group	age	english.grade	math.grade
0	31.31	120.62	F	NaN	22	3.5	3.7
1	34.39	-118.54	M	NaN	22	2.9	3.2
2	37.80	-122.27	F	NaN	22	3.9	3.8
3	37.69	-122.09	F	NaN	20	3.3	2.8
4	-23.18	-45.88	F	NaN	21	3.7	2.6

	sciences.grade	language.grade	portfolio.rating	coverletter.rating
0	3.1	1.0	4	4.0
1	3.6	5.0	5	4.0
2	3.2	5.0	3	3.0
3	3.2	5.0	5	2.0
4	3.4	1.0	4	4.0

	refletter.rating
0	4
1	5
2	4
3	4
4	5

12. Read Excel File

Create an Excel file named `students.xlsx` containing student records. Read the Excel file using Pandas and display the DataFrame. Display the column names and shape of the DataFrame.

```
# datafile:- https://github.com/SanyaShresta25/student-registration-system/blob/main/Student_data.xlsx
url = 'Student_data.xlsx'
student = pd.read_excel(url)
print(student)
```

	Registration No.	Name	Class	Gender	DOB
0	1	Sanya Shresta	5	2	25/04/2004
1	2	Saanvi U.	5	1	02/03/2004
2	3	Laila	6	1	01/01/2003
3	4	Amila	6	1	23/04/2007
4	5	Bravo Mitchell	10	1	23/04/1999
5	6	Bella	8	1	23/05/2004
6	7	Nidhi	8	2	02/02/2004
7	8	Meena	9	2	23/09/2004
8	9	Pinky	4	1	12/09/2004
9	10	Sayeem	4	1	12/03/2004
10	11	Nishan	9	1	23/09/2006

	Date of Registration	Religion	Skill	Father Name	Mother Name
0	16/04/2024	Christian	Python	Deeshan	Deeksha
1	16/04/2024	Hindu	Java	Niranjan	Naina
2	16/04/2024	Muslim	C	Ahad	Insha
3	16/04/2024	Hindu	Python Dev	Kishan	Smrithi
4	16/04/2024	Christian	Cricketer	Gary	Lena
5	16/04/2024	Hindu	C	Codd	Anna
6	17/04/2024	Hindu	Python	Papa	Amma
7	17/04/2024	Hindu	Python	deeshan	deeksha
8	17/04/2024	Hindu	Node.js	Kushagr	No
9	17/04/2024	Muslim	Everything	abba	ammi
10	18/04/2024	Christian	mySQL	Vedang	Hema

	Father's Occupation	Mother's Occupation
0	Engineer	Homemaker
1	Financial Adviser	Homemaker
2	SDE	SDE
3	Cricketer	Lecturer
4	TV reporter	Singer
5	SDE	SdE
6	Kelsa	Kelsa
7	Engineer	homemaker
8	Vela	mafia
9	sde	sde
10	ML specialist	Homemaker

13. Setting Index

Using the `students` DataFrame, set the `RollNo` column as the index. Display the DataFrame and access the record of a student using the index label.

```
student=student.set_index("Registration No.")
print(student.loc[2])
```

```
level_0          1
index            1
Name            Saanvi U.
Class            5
Gender            1
DOB            02/03/2004
Date of Registration  16/04/2024
Religion          Hindu
Skill             Java
Father Name       Niranjan
Mother Name       Naina
Father's Occupation  Financial Adviser
Mother's Occupation  Homemaker
Name: 2, dtype: object
```

14. Resetting Index

Reset the index of the DataFrame created in the previous question. Demonstrate both keeping the old index as a column and removing the old index.

```
student=student.reset_index()
```

15. Integrated Pandas Exercise

Create a student DataFrame containing RollNo, Name, Department, Marks, and City. Set RollNo as the index, select students with Marks >= 75, reset the index, and finally save the resulting DataFrame as both CSV and Excel files.

```
student_data={'rollno':[1,2,3,4],
              'name':['yatharth','sahil','vimal','akshay'],
              'department':['CE','MECH','CE','MECH'],
              'marks':[12,34,56,78],
              'city':['rajkot','jamnagar','rajkot','dhandhuka']}
student=pd.DataFrame(student_data)
student=student.set_index('rollno')
res=student[student['marks']>=75]
print("marks greater than 75\n",res)
student=student.reset_index()
res.to_csv("top_student.csv")
res.to_excel("top_student.xlsx")
```

```
marks greater than 75
   name department  marks    city
rollno
4    akshay      MECH    78 dhandhuka
```



Python for Data Science - 2305CS303

Lab - 11

Roll No. : 122

Name : Yatharth Sanghavi

```
import pandas as pd
```

1. Create a MultiIndexed DataFrame

Create a DataFrame containing student marks for different Departments and Semesters. Use `set_index()` to create a MultiIndex using `Department` and `Semester`. Display the DataFrame and its index.

```
student_data={'rollno':[1,2,3,4],
              'name':['yatharth','sahil','vimal','akshay'],
              'department':['CE','MECH','CE','MECH'],
              'semester':[2,3,4,5],
              'marks':[12,34,56,78],
              'city':['rajkot','jamnagar','rajkot','dhandhuka']}
student = pd.DataFrame(student_data)
student.set_index(['department','semester'])
```

		rollno	name	marks	city
department	semester				
CE	2	1	yatharth	12	rajkot
MECH	3	2	sahil	34	jamnagar
CE	4	3	vimal	56	rajkot
MECH	5	4	akshay	78	dhandhuka

2. Create a MultiIndex using Multiple Columns

Create a DataFrame containing `Department`, `Year`, `Gender`, `Student` and `Marks`. Set `Department`, `Year` and `Gender` as the MultiIndex and display the resulting DataFrame.

```
student_data={'rollno':[1,2,3,4],
              'name':['yatharth','sahil','vimal','akshay'],
              'department':['CE','MECH','CE','MECH'],
              'semester':[2,3,4,5],
              'marks':[12,34,56,78],
              'year':[20,21,22,23],
              'gender':['Male','Male','Male','Male'],
              'city':['rajkot','jamnagar','rajkot','dhandhuka']}
student = pd.DataFrame(student_data)
student.set_index(['department','year','gender'])
```

			rollno	name	semester	marks	city
department	year	gender					
CE	20	Male	1	yatharth	2	12	rajkot
MECH	21	Male	2	sahil	3	34	jamnagar
CE	22	Male	3	vimal	4	56	rajkot
MECH	23	Male	4	akshay	5	78	dhandhuka

3. Read a MultiIndexed DataFrame Directly from CSV

Create a CSV file containing columns `Department`, `Semester`, `Subject`, and `Marks`. Read the CSV file directly as a MultiIndexed DataFrame using `pd.read_csv()` with the appropriate `index_col` parameter. Display the resulting DataFrame.

```
csv_data = """Department,Semester,Subject,Marks
CE,2,Python,85
CE,2,Maths,90
MECH,3,Thermodynamics,78
MECH,3,Fluid Mechanics,82
"""

with open("student_marks.csv", "w") as f:
    f.write(csv_data)

df_csv = pd.read_csv("student_marks.csv", index_col=["Department", "Semester"])
print("MultiIndexed DataFrame from CSV:")
display(df_csv)
```

MultiIndexed DataFrame from CSV:

		Subject	Marks
Department	Semester		
CE	2	Python	85
	2	Maths	90
MECH	3	Thermodynamics	78
	3	Fluid Mechanics	82

4. Inspect a MultiIndexed DataFrame

Using the MultiIndexed DataFrame created in Question 3, display the index levels, index names, columns, shape, and data types.

```
print("Index Levels:", df_csv.index.levels)
print("\nIndex Names:", df_csv.index.names)
print("\nColumns:", df_csv.columns.tolist())
print("\nShape:", df_csv.shape)
print("\nData Types:\n", df_csv.dtypes)
```

```
Index Levels: [['CE', 'MECH'], [2, 3]]
Index Names: ['Department', 'Semester']
Columns: ['Subject', 'Marks']
Shape: (4, 2)
Data Types:
Subject    object
Marks      int64
dtype: object
```

5. Access Data using MultiIndex

Using a MultiIndexed DataFrame having `Department` and `Semester` as indexes, access all records belonging to one department and one specific department-semester combination.

```
print("--- Records for 'CE' Department ---")
print(df_csv.loc["CE"])

print("\n--- Records for 'CE', Semester 2 ---")
print(df_csv.loc[("CE", 2)])
```

```

--- Records for 'CE' Department ---
      Subject  Marks
Semester
2         Python    85
2         Maths    90

--- Records for 'CE', Semester 2 ---
      Subject  Marks
Department Semester
CE          2     Python    85
CE          2     Maths    90

```

6. Cross Section using xs()

Create a MultIndexed DataFrame with `Department` and `Semester` as index levels. Use `xs()` to retrieve the records for a particular semester across all departments.

```

print("--- Cross-section for Semester 2 across all departments ---")
print(df_csv.xs(key=2, level="Semester"))

```

```

--- Cross-section for Semester 2 across all departments ---
      Subject  Marks
Department
CE         Python    85
CE         Maths    90

```

7. Cross Section from a Specific Level

Using the same MultIndexed DataFrame, use `xs()` with the `level` parameter to retrieve records for one department. Display the result.

```

print("--- Cross-section for 'CE' Department ---")
print(df_csv.xs(key="CE", level="Department"))

```

```

--- Cross-section for 'CE' Department ---
      Subject  Marks
Semester
2         Python    85
2         Maths    90

```

8. Cross Section on Multiple Index Levels

Create a MultIndexed DataFrame with `Department`, `Semester`, and `Subject` as index levels. Use `xs()` to retrieve the records corresponding to a particular combination of Department and Semester.

```

df_3level = pd.read_csv(
    "student_marks.csv", index_col=["Department", "Semester", "Subject"]
)
print(df_3level.xs(key=("CE", 2), level=["Department", "Semester"]))

```

```

      Marks
Subject
Python    85
Maths     90

```

9. Identify Missing Data

Create a student DataFrame containing `Name`, `Department`, `Marks`, `Attendance`, and `City`, with some missing values represented by `NaN` or `None`. Display the DataFrame and identify the missing values using `isna()`.

```

import numpy as np
data_missing = {
    "Name": ["Yatharth", "Sahil", "Vimal", "Akshay"],
    "Department": ["CE", "MECH", "CE", None],
    "Marks": [85, np.nan, 90, 78],
    "Attendance": [92, 80, np.nan, 88],
    "City": ["Rajkot", "Jamnagar", None, "Dhandhuka"],
}
df_student = pd.DataFrame(data_missing)

print("Original DataFrame:")
display(df_student)

```

```
print("\nMissing values identification using isna():")
display(df_student.isna())
```

Original DataFrame:

	Name	Department	Marks	Attendance	City
0	Yatharth	CE	85.0	92.0	Rajkot
1	Sahil	MECH	NaN	80.0	Jamnagar
2	Vimal	CE	90.0	NaN	None
3	Akshay	None	78.0	88.0	Dhandhuka

Missing values identification using isna():

	Name	Department	Marks	Attendance	City
0	False	False	False	False	False
1	False	False	True	False	False
2	False	False	False	True	True
3	False	True	False	False	False

10. Count Missing Values

Using the DataFrame from Question 9, calculate the number of missing values in each column and display the total number of missing values in the complete DataFrame.

```
print("Missing values count per column:")
print(df_student.isna().sum())

print(
    "\nTotal missing values in complete DataFrame:", df_student.isna().sum().sum()
)
```

Missing values count per column:

```
Name      0
Department 1
Marks      1
Attendance 1
City       1
dtype: int64
```

Total missing values in complete DataFrame: 4

11. Detect Missing Values using isnull()

Create a DataFrame containing missing values in different columns. Use `isnull()` and `notnull()` to display the missing and non-missing values. Compare the results of `isnull()` and `isna()`.

```
print("Using isnull():")
display(df_student.isnull())

print("\nUsing notnull():")
display(df_student.notnull())

are_identical = df_student.isnull().equals(df_student.isna())
print(
    f"\nAre isnull() and isna() identical? {are_identical} (isna is an alias for isnull)"
)
```

Using isnull():

	Name	Department	Marks	Attendance	City
0	False	False	False	False	False
1	False	False	True	False	False
2	False	False	False	True	True
3	False	True	False	False	False

Using notnull():

	Name	Department	Marks	Attendance	City
0	True	True	True	True	True
1	True	True	False	True	True
2	True	True	True	False	False
3	True	False	True	True	True

Are isnull() and isna() identical? True (isna is an alias for isnull)

12. Remove Missing Values

Using a DataFrame containing missing values, demonstrate `dropna()` to remove rows containing at least one missing value. Also demonstrate how to remove columns containing missing values.

```
print("--- DataFrame after dropping rows with any missing value ---")
display(df_student.dropna(axis=0))

print("\n--- DataFrame after dropping columns with any missing value ---")
display(df_student.dropna(axis=1))
```

--- DataFrame after dropping rows with any missing value ---

	Name	Department	Marks	Attendance	City
0	Yatharth	CE	85.0	92.0	Rajkot

--- DataFrame after dropping columns with any missing value ---

	Name
0	Yatharth
1	Sahil
2	Vimal
3	Akshay

13. Fill Missing Values with a Constant

Create a DataFrame containing missing values in the `Marks` and `City` columns. Replace missing values using suitable constant values with `fillna()`.

```
df_filled_const = df_student.fillna({"Marks": 0, "City": "Unknown"})

print("DataFrame after filling missing values with constants:")
display(df_filled_const)
```

DataFrame after filling missing values with constants:

	Name	Department	Marks	Attendance	City
0	Yatharth	CE	85.0	92.0	Rajkot
1	Sahil	MECH	0.0	80.0	Jamnagar
2	Vimal	CE	90.0	NaN	Unknown
3	Akshay	None	78.0	88.0	Dhandhuka

14. Fill Missing Values using Mean and Median

Create a DataFrame containing missing numerical values. Fill missing values in the `Marks` column using the mean and demonstrate filling another numerical column using the median.

```
df_num_filled = df_student.copy()

mean_marks = df_num_filled["Marks"].mean()
df_num_filled["Marks"] = df_num_filled["Marks"].fillna(mean_marks)

median_att = df_num_filled["Attendance"].median()
df_num_filled["Attendance"] = df_num_filled["Attendance"].fillna(median_att)

print(
    f"Mean Marks used: {mean_marks:.2f} | Median Attendance used: {median_att:.2f}"
)
display(df_num_filled)
```

Mean Marks used: 84.33 | Median Attendance used: 88.00

	Name	Department	Marks	Attendance	City
0	Yatharth	CE	85.000000	92.0	Rajkot
1	Sahil	MECH	84.333333	80.0	Jamnagar
2	Vimal	CE	90.000000	88.0	None
3	Akshay	None	78.000000	88.0	Dhandhuka

15. Fill Missing Categorical Values using Mode

Create a DataFrame containing missing values in a categorical column such as `City` or `Department`. Fill the missing values using the mode of that column.

```
df_cat_filled = df_student.copy()

dept_mode = df_cat_filled["Department"].mode()[0]
df_cat_filled["Department"] = df_cat_filled["Department"].fillna(dept_mode)

city_mode = df_cat_filled["City"].mode()[0]
df_cat_filled["City"] = df_cat_filled["City"].fillna(city_mode)

print(f"Department Mode: {dept_mode} | City Mode: {city_mode}")
display(df_cat_filled)
```

Department Mode: CE | City Mode: Dhandhuka

	Name	Department	Marks	Attendance	City
0	Yatharth	CE	85.0	92.0	Rajkot
1	Sahil	MECH	NaN	80.0	Jamnagar
2	Vimal	CE	90.0	NaN	Dhandhuka
3	Akshay	CE	78.0	88.0	Dhandhuka

16. Forward Fill and Backward Fill

Create a time-series-like DataFrame containing missing values. Demonstrate forward filling using `ffill()` and backward filling using `bfill()`. Observe the difference between the two methods.

```
ts_data = {
    "Date": pd.date_range(start="2024-01-01", periods=5),
    "Reading": [10.0, np.nan, np.nan, 25.0, np.nan],
}
df_ts = pd.DataFrame(ts_data)

print("\nForward Fill (ffill()):")
display(df_ts.fffll())

print("\nBackward Fill (bfill()):")
display(df_ts.bfill())
```

Forward Fill (ffill()):

	Date	Reading
0	2024-01-01	10.0
1	2024-01-02	10.0
2	2024-01-03	10.0
3	2024-01-04	25.0
4	2024-01-05	25.0

Backward Fill (bfill()):

	Date	Reading
0	2024-01-01	10.0
1	2024-01-02	25.0
2	2024-01-03	25.0
3	2024-01-04	25.0
4	2024-01-05	NaN

17. Groupby - Basic Grouping

Create a DataFrame containing `Department`, `Student`, and `Marks`. Group the data by `Department` and calculate the average marks for each department.

```
df_grp = pd.DataFrame(
    {
        "Department": ["CE", "MECH", "CE", "MECH", "CE"],
        "Student": ["Yatharth", "Sahil", "Vimal", "Akshay", "Rohan"],
        "Marks": [85, 78, 90, 82, 88],
    }
)

avg_marks = df_grp.groupby("Department")["Marks"].mean()
print("Average Marks by Department:")
print(avg_marks)
```

```
Average Marks by Department:
Department
CE      87.666667
MECH    80.000000
Name: Marks, dtype: float64
```

18. Groupby with Multiple Aggregations

Using a student DataFrame, group the records by `Department` and calculate the minimum, maximum, mean, and count of `Marks` for each department.

```
agg_results = df_grp.groupby("Department")["Marks"].agg(
    ["min", "max", "mean", "count"]
)
print("Multiple Aggregations on Marks by Department:")
display(agg_results)
```

```
Multiple Aggregations on Marks by Department:
```

	min	max	mean	count
Department				
CE	85	90	87.666667	3
MECH	78	82	80.000000	2

19. Groupby with Multiple Columns

Create a DataFrame containing `Department`, `Semester`, `Student`, and `Marks`. Group the data by both `Department` and `Semester` and calculate the average marks.

```
df_multi_grp = pd.DataFrame(
    {
        "Department": ["CE", "CE", "MECH", "MECH", "CE"],
```

```

        "Semester": [2, 2, 3, 3, 4],
        "Student": ["Yatharth", "Aman", "Sahil", "Akshay", "Vimal"],
        "Marks": [85, 75, 78, 82, 90],
    }
)

avg_by_dept_sem = df_multi_grp.groupby(["Department", "Semester"])[
    "Marks"
].mean()
print("Average Marks grouped by Department and Semester:")
print(avg_by_dept_sem)

```

Average Marks grouped by Department and Semester:

Department	Semester	Marks
CE	2	80.0
	4	90.0
MECH	3	80.0

Name: Marks, dtype: float64

20. Named Aggregation using agg()

Using a student DataFrame, group the data by `Department` and use `agg()` with named aggregation to calculate `Maximum_Marks`, `Minimum_Marks`, and `Average_Marks`.

```

named_agg = df_grp.groupby("Department").agg(
    Maximum_Marks=("Marks", "max"),
    Minimum_Marks=("Marks", "min"),
    Average_Marks=("Marks", "mean"),
)

print("Named Aggregation Results:")
display(named_agg)

```

Named Aggregation Results:

	Maximum_Marks	Minimum_Marks	Average_Marks
Department			
CE	90	85	87.666667
MECH	82	78	80.000000

Next steps: [Generate code with named_agg](#)

[New interactive sheet](#)

21. Groupby and Count Records

Create a DataFrame containing student details and departments. Use `groupby()` to count the number of students in each department. Also calculate the number of unique cities for each department.

```

df_details = pd.DataFrame(
    {
        "Student": ["Yatharth", "Sahil", "Vimal", "Akshay", "Rohan"],
        "Department": ["CE", "MECH", "CE", "MECH", "CE"],
        "City": ["Rajkot", "Jamnagar", "Rajkot", "Dhandhuka", "Surat"],
    }
)

dept_summary = df_details.groupby("Department").agg(
    Student_Count=("Student", "count"), Unique_Cities=("City", "nunique")
)

print("Department summary - Student Count and Unique Cities:")
display(dept_summary)

```

Department summary - Student Count and Unique Cities:

	Student_Count	Unique_Cities
Department		
CE	3	2
MECH	2	2

Next steps: [Generate code with dept_summary](#)

[New interactive sheet](#)

22. Groupby after Handling Missing Data

Create a DataFrame containing `Department`, `Marks`, and `City` with missing values. Handle the missing marks appropriately and then group the data by department to calculate average marks.

```
df_missing_grp = pd.DataFrame(
    {
        "Department": ["CE", "MECH", "CE", "MECH", "CE"],
        "Marks": [85, np.nan, 90, 82, np.nan],
        "City": ["Rajkot", "Jamnagar", None, "Dhandhuka", "Rajkot"],
    }
)

print("Original Data with missing values:")
display(df_missing_grp)

overall_mean = df_missing_grp["Marks"].mean()
df_missing_grp["Marks"] = df_missing_grp["Marks"].fillna(overall_mean)

dept_avg = df_missing_grp.groupby("Department")["Marks"].mean()

print("\nAverage Marks by Department (after replacing missing values):")
print(dept_avg)
```

Original Data with missing values:

	Department	Marks	City	
0	CE	85.0	Rajkot	
1	MECH	NaN	Jamnagar	
2	CE	90.0	None	



Python for Data Science - 2305CS303

Lab - 12

Roll No. : 122

Name : Yatharth Sanghavi

1. Concatenate DataFrames Row-wise

Create two DataFrames containing student details from two different groups. Use `pd.concat()` with `axis=0` to concatenate the DataFrames row-wise. Display the original DataFrames and the resulting DataFrame.

```
import pandas as pd

group1 = pd.DataFrame({
    "RollNo": [1, 2, 3],
    "Name": ["Amit", "Riya", "Karan"],
    "City": ["Rajkot", "Surat", "Vadodara"]
})
group2 = pd.DataFrame({
    "RollNo": [4, 5, 6],
    "Name": ["Neha", "Raj", "Pooja"],
    "City": ["Ahmedabad", "Rajkot", "Surat"]
})

print("Group 1")
print(group1)
print("\nGroup 2")
print(group2)

result = pd.concat([group1, group2], axis=0)
print("\nConcatenated (row-wise)")
print(result)
```

```
Group 1
  RollNo  Name  City
0      1  Amit  Rajkot
1      2  Riya  Surat
2      3  Karan  Vadodara
```

```
Group 2
  RollNo  Name  City
0      4  Neha  Ahmedabad
1      5   Raj  Rajkot
2      6 Pooja   Surat
```

```
Concatenated (row-wise)
  RollNo  Name  City
0      1  Amit  Rajkot
1      2  Riya  Surat
2      3  Karan  Vadodara
0      4  Neha  Ahmedabad
1      5   Raj  Rajkot
2      6 Pooja   Surat
```

2. Concatenate DataFrames Column-wise

Create two DataFrames containing student personal details and academic details with the same index. Use `pd.concat()` with `axis=1` to combine the DataFrames column-wise. Display the resulting DataFrame.

```
personal = pd.DataFrame({
    "Name": ["Amit", "Riya", "Karan"],
    "City": ["Rajkot", "Surat", "Vadodara"]
}, index=[1, 2, 3])
academic = pd.DataFrame({
    "Marks": [78, 85, 72],
    "Grade": ["B", "A", "C"]
}, index=[1, 2, 3])

result = pd.concat([personal, academic], axis=1)
print(result)
```

	Name	City	Marks	Grade
1	Amit	Rajkot	78	B
2	Riya	Surat	85	A
3	Karan	Vadodara	72	C

3. Concatenate DataFrames with Different Columns

Create two DataFrames containing student records with some common and some different columns. Use `pd.concat()` with `axis=0` and observe how pandas handles columns that are not present in both DataFrames.

```
df1 = pd.DataFrame({
    "RollNo": [1, 2, 3],
    "Name": ["Amit", "Riya", "Karan"],
    "City": ["Rajkot", "Surat", "Vadodara"]
})
df2 = pd.DataFrame({
    "RollNo": [4, 5, 6],
    "Name": ["Neha", "Raj", "Pooja"],
    "Marks": [80, 75, 90]
})

result = pd.concat([df1, df2], axis=0)
print(result)
# Columns not present in both DataFrames (City, Marks) are filled with NaN
```

	RollNo	Name	City	Marks
0	1	Amit	Rajkot	NaN
1	2	Riya	Surat	NaN
2	3	Karan	Vadodara	NaN
0	4	Neha	NaN	80.0
1	5	Raj	NaN	75.0
2	6	Pooja	NaN	90.0

4. Concatenate DataFrames and Reset the Index

Create two DataFrames representing student records from two classes, each having its own index. Concatenate them row-wise using `pd.concat()` and use `ignore_index=True` to generate a new continuous index.

```
class1 = pd.DataFrame({
    "RollNo": [1, 2, 3],
    "Name": ["Amit", "Riya", "Karan"]
}, index=[0, 1, 2])
class2 = pd.DataFrame({
    "RollNo": [4, 5, 6],
    "Name": ["Neha", "Raj", "Pooja"]
}, index=[0, 1, 2])

print("Without ignore_index")
print(pd.concat([class1, class2], axis=0))

print("\nWith ignore_index=True")
print(pd.concat([class1, class2], axis=0, ignore_index=True))
```

	RollNo	Name
0	1	Amit
1	2	Riya
2	3	Karan
0	4	Neha

```

1      5      Raj
2      6      Pooja

With ignore_index=True
RollNo  Name
0       1    Amit
1       2    Riya
2       3    Karan
3       4    Neha
4       5      Raj
5       6    Pooja

```

5. Concatenate Multiple DataFrames

Create three DataFrames containing student records from three semesters. Use `pd.concat()` to combine all three DataFrames row-wise. Display the final DataFrame and its shape.

```

sem1 = pd.DataFrame({"RollNo": [1, 2], "Name": ["Amit", "Riya"], "Semester": [1, 1]})
sem2 = pd.DataFrame({"RollNo": [1, 2], "Name": ["Amit", "Riya"], "Semester": [2, 2]})
sem3 = pd.DataFrame({"RollNo": [1, 2], "Name": ["Amit", "Riya"], "Semester": [3, 3]})

result = pd.concat([sem1, sem2, sem3], axis=0, ignore_index=True)
print(result)
print("Shape:", result.shape)

```

```

RollNo  Name  Semester
0       1    Amit       1
1       2    Riya       1
2       1    Amit       2
3       2    Riya       2
4       1    Amit       3
5       2    Riya       3
Shape: (6, 3)

```

6. Join Two DataFrames using Index

Create two DataFrames containing student information and marks, with `RollNo` as the index. Use the `join()` method to combine the DataFrames based on their index. Display the resulting DataFrame.

```

student_info = pd.DataFrame({
    "Name": ["Amit", "Riya", "Karan"],
    "City": ["Rajkot", "Surat", "Vadodara"]
}, index=pd.Index([101, 102, 103], name="RollNo"))

marks = pd.DataFrame({
    "Python": [78, 85, 72],
    "DBMS": [80, 90, 65]
}, index=pd.Index([101, 102, 103], name="RollNo"))

result = student_info.join(marks)
print(result)

```

```

      Name      City  Python  DBMS
RollNo
101    Amit   Rajkot     78    80
102    Riya    Surat     85    90
103    Karan  Vadodara     72    65

```

7. Join DataFrames with Different Index Values

Create two DataFrames containing student details and attendance information with partially matching indexes. Demonstrate `join()` using the default left join and then using `how='inner'`. Compare the results.

```

details = pd.DataFrame({
    "Name": ["Amit", "Riya", "Karan", "Neha"],
    "City": ["Rajkot", "Surat", "Vadodara", "Ahmedabad"]
}, index=pd.Index([101, 102, 103, 104], name="RollNo"))

attendance = pd.DataFrame({
    "Attendance": [90, 85, 70]
}, index=pd.Index([101, 102, 105], name="RollNo"))

print("Left join (default)")
print(details.join(attendance))

print("\nInner join")
print(details.join(attendance, how="inner"))

```

```
# Left join keeps all rows of the left DataFrame (NaN where no match),
# inner join keeps only the RollNo values present in both.
```

```
Left join (default)
      Name      City  Attendance
RollNo
101    Amit    Rajkot      90.0
102    Riya    Surat      85.0
103    Karan  Vadodara      NaN
104    Neha  Ahmedabad      NaN
```

```
Inner join
      Name      City  Attendance
RollNo
101    Amit  Rajkot      90
102    Riya  Surat      85
```

8. Merge DataFrames using a Common Column

Create two DataFrames containing student details and examination marks. Both DataFrames should contain a common column named `RollNo`. Use `pd.merge()` to combine them based on `RollNo` and display the result.

```
student = pd.DataFrame({
    "RollNo": [1, 2, 3, 4],
    "Name": ["Amit", "Riya", "Karan", "Neha"]
})
exam = pd.DataFrame({
    "RollNo": [1, 2, 3, 4],
    "Marks": [78, 85, 72, 90]
})

result = pd.merge(student, exam, on="RollNo")
print(result)
```

```
RollNo  Name  Marks
0      1  Amit    78
1      2  Riya    85
2      3  Karan    72
3      4  Neha    90
```

9. Merge DataFrames using Different Join Types

Create two DataFrames containing student records from two different sources with some common and some non-common `RollNo` values. Use `pd.merge()` to demonstrate `inner`, `left`, `right`, and `outer` joins. Display and compare all four results.

```
source1 = pd.DataFrame({
    "RollNo": [1, 2, 3, 4],
    "Name": ["Amit", "Riya", "Karan", "Neha"]
})
source2 = pd.DataFrame({
    "RollNo": [3, 4, 5, 6],
    "Marks": [72, 90, 80, 65]
})

print("Inner join")
print(pd.merge(source1, source2, on="RollNo", how="inner"))

print("\nLeft join")
print(pd.merge(source1, source2, on="RollNo", how="left"))

print("\nRight join")
print(pd.merge(source1, source2, on="RollNo", how="right"))

print("\nOuter join")
print(pd.merge(source1, source2, on="RollNo", how="outer"))
```

```
Inner join
RollNo  Name  Marks
0      3  Karan    72
1      4  Neha    90
```

```
Left join
RollNo  Name  Marks
0      1  Amit   NaN
1      2  Riya   NaN
2      3  Karan  72.0
3      4  Neha  90.0
```

```
Right join
RollNo  Name  Marks
0      3  Karan    72
```

1	4	Neha	90
2	5	NaN	80
3	6	NaN	65

Outer join

	RollNo	Name	Marks
0	1	Amit	NaN
1	2	Riya	NaN
2	3	Karan	72.0
3	4	Neha	90.0
4	5	NaN	80.0
5	6	NaN	65.0

10. Combine Student Academic Data using Concat, Join and Merge

Create three DataFrames containing student personal information, semester marks, and attendance information. Use `pd.concat()` to combine two suitable DataFrames, `join()` to combine data using an index, and `pd.merge()` to combine data using a common column. Display the final combined student dataset.

```
# Personal information of students (two batches)
personal_a = pd.DataFrame({
    "RollNo": [1, 2],
    "Name": ["Amit", "Riya"],
    "City": ["Rajkot", "Surat"]
})
personal_b = pd.DataFrame({
    "RollNo": [3, 4],
    "Name": ["Karan", "Neha"],
    "City": ["Vadodara", "Ahmedabad"]
})

# Semester marks (RollNo as index)
marks = pd.DataFrame({
    "Sem1": [70, 80, 65, 90],
    "Sem2": [75, 85, 60, 88]
}, index=pd.Index([1, 2, 3, 4], name="RollNo"))

# Attendance (RollNo as a column)
attendance = pd.DataFrame({
    "RollNo": [1, 2, 3, 4],
    "Attendance": [92, 88, 75, 95]
})

# Step 1: concat -> combine the two personal DataFrames
personal = pd.concat([personal_a, personal_b], ignore_index=True)

# Step 2: join -> add marks using the index
personal_indexed = personal.set_index("RollNo")
with_marks = personal_indexed.join(marks)
with_marks = with_marks.reset_index()

# Step 3: merge -> add attendance using the common column RollNo
final = pd.merge(with_marks, attendance, on="RollNo")

print("Final combined student dataset")
print(final)
```

Final combined student dataset

	RollNo	Name	City	Sem1	Sem2	Attendance
0	1	Amit	Rajkot	70	75	92
1	2	Riya	Surat	80	85	88
2	3	Karan	Vadodara	65	60	75
3	4	Neha	Ahmedabad	90	88	95



Python for Data Science (PDS)

Lab - 13

Roll No. : 122

Name : Yatharth

1. Fetch MCA Faculty Page and Check Status

Send an HTTP GET request to <https://darshan.ac.in/faculty-list/mca> using requests.get().

Display the HTTP status code to verify a successful connection (200) and print the length of the page source code.

```
import requests
res= requests.get('https://darshan.ac.in/faculty-list/mca')
print(res.status_code)
print(len(res.text))
```

```
200
167088
```

2. Extract Web Page Title.

Parse the HTML content using BeautifulSoup with 'html.parser'.

Find and display the content of the title tag to identify the webpage.

```
from bs4 import BeautifulSoup
soup = BeautifulSoup(res.text,"html.parser")
print(soup.title.text)
```

```
Faculty List | MCA | Darshan University, Rajkot
```

3. Scrape Faculty Names using Heading Tags.

Locate all faculty names displayed on the page (typically enclosed in h3 tag or specific heading tags within faculty cards).

Loop through the results and display the first 5 faculty names cleanly without leading or trailing spaces

```
name = soup.find_all('h2','g-mb-0')
names = []
for i in name:
    names.append(i.text.strip())
print(names)
```

```
['Dr. Gopi Sanghani', 'Prof. Swati Sharma', 'Prof. Maulik Trivedi', 'Prof. Firoz Sherasiya', 'Prof. Mayur Padia', 'Prof. Vas
```

4. Extract Faculty Designations.

Identify the HTML tag or class used for designations (e.g., Assistant Professor, Associate Professor, Head of Department).

Extract and print the designations of the first 5 faculty members.

```
designation=soup.find_all('span','g-mb-10')
designations=[]
for i in designation:
    designations.append(i.text.strip())
print(designations)
```

```
['Dean - Faculty of Computer Application', 'Assistant Professor', 'Assistant Professor', 'Assistant Professor', 'Assistant F
```

5. Scrape Faculty Qualifications.

Find the tags or paragraph elements containing qualification details (such as Ph.D., M.Phil, MCA, M.Tech) for faculty members.

Iterate through them and print the qualification strings.

```
qualification=soup.select('span.g-px-20')
qualifications=[]
for i in qualification:
    qualifications.append(i.text.strip())
print(qualifications)
```

```
['Ph.D. , M.E. (CE)', 'PhD(Pursuing), M.Tech. (CSE)', 'Ph.D.(Pursuing), M.Tech.(CSE)', 'Ph.D.(Pursuing), M.E. (CE)', 'M.E. (
```

6. Extract Profile Page Links using CSS Selectors

Use soup.select() to find anchor tags leading to individual faculty detail profiles.

Extract and print both the faculty member's name (link text) and their profile URL (href).

```
profile = soup.select('a.g-brd-bottom')
profiles=[]
for i in profile:
    profiles.append(f'{i["host"]}{i["href"]}')
profiles
```

```
['https://darshan.ac.in/faculty-profile/Dr-Gopi-Bhupendrabhai-Sanghani',
'https://darshan.ac.in/faculty-profile/Prof-Swati-Rameshbhai-Sharma',
'https://darshan.ac.in/faculty-profile/Prof-Maulik-D-Trivedi',
'https://darshan.ac.in/faculty-profile/Prof-Firoz-Ahamadbhai-Sherasiya',
'https://darshan.ac.in/faculty-profile/Prof-Mayur-Shashikant-Padia',
'https://darshan.ac.in/faculty-profile/Prof-Vaseem-Jusabbhai-Ghada',
'https://darshan.ac.in/faculty-profile/Prof-Umeshkumar-Hemubhai-Thoriya',
'https://darshan.ac.in/faculty-profile/Prof-Mehul-Dilipbhai-Bhundiya',
'https://darshan.ac.in/faculty-profile/Prof-Jayesh-Dhirubhai-Vagadiya',
'https://darshan.ac.in/faculty-profile/Prof-Rajkumar-Batukbhai-Gondaliya',
'https://darshan.ac.in/faculty-profile/Prof-Shruti-Rajeshbhai-Maniar',
'https://darshan.ac.in/faculty-profile/Prof-Dharmik-Pareshbhai-Vasiyani',
'https://darshan.ac.in/faculty-profile/Prof-Devangi-Lalitbhai-Kotak',
'https://darshan.ac.in/faculty-profile/Prof-Madhuresh-Rajeshbhai-Fichadiya']
```

7. Extract Faculty Profile Image URLs.

Locate all (img) tags representing faculty photos inside the card containers.

Loop through them and print the image source URL (src) and the alt text attribute for each faculty member.

```
profile_img = soup.select('img.img-faculty')
profiles_img=[]
for i in profile_img:
    profiles_img.append(i['src'])
profiles_img
```

```
['https://du-website.s3.ap-south-1.amazonaws.com/U01/Faculty-Photo/5---29-04-2023-11-00-29.jpg',
'https://du-website.s3.ap-south-1.amazonaws.com/U01/Faculty-Photo/11---29-04-2023-01-44-21.jpg',
```

```
'https://du-website.s3.ap-south-1.amazonaws.com/U01/Faculty-Photo/8---28-04-2023-02-06-25.jpg',
'https://du-website.s3.ap-south-1.amazonaws.com/U01/Faculty-Photo/12---28-04-2023-02-06-51.jpg',
'https://du-website.s3.ap-south-1.amazonaws.com/U01/Faculty-Photo/13---28-04-2023-02-07-48.jpg',
'https://du-website.s3.ap-south-1.amazonaws.com/U01/Faculty-Photo/317---28-04-2023-01-32-29.jpg',
'https://du-website.s3.ap-south-1.amazonaws.com/U01/Faculty-Photo/221---28-04-2023-02-08-50.jpg',
'https://du-website.s3.ap-south-1.amazonaws.com/U01/Faculty-Photo/294---28-04-2023-02-09-43.jpg',
'https://du-website.s3.ap-south-1.amazonaws.com/U01/Faculty-Photo/219---28-04-2023-02-09-01.jpg',
'https://du-website.s3.ap-south-1.amazonaws.com/U01/Faculty-Photo/263---09-05-2023-01-10-49.jpg',
'https://du-website.s3.ap-south-1.amazonaws.com/U01/Faculty-Photo/336---28-04-2023-01-34-37.jpg',
'https://du-website.s3.ap-south-1.amazonaws.com/U01/Faculty-Photo/335---27-01-2025-09-33-28.jpg',
'https://du-website.s3.ap-south-1.amazonaws.com/U01/Faculty-Photo/359---29-04-2023-01-45-32.jpg',
'https://du-website.s3.ap-south-1.amazonaws.com/U01/Faculty-Photo/331---28-04-2023-01-33-39.jpg']
```

8. Filter Faculty by Specific Designation or Qualification.

Loop through all scraped faculty designations/qualifications and filter only those holding a specific title (e.g., "Assistant Professor" or containing "Ph.D.").

Store and display the filtered results in a Python list.

```
filter = []
for i in zip(names, qualifications):
    if i[1] == 'Assistant Professor' or 'Ph.D.' in i[1]:
        print(i)

('Dr. Gopi Sanghani', 'Ph.D. , M.E. (CE)')
('Prof. Maulik Trivedi', 'Ph.D.(Pursuing), M.Tech.(CSE)')
('Prof. Firoz Sherasiya', 'Ph.D.(Pursuing), M.E. (CE)')
('Prof. Vaseem Ghada', 'Ph.D. (Pursuing), M.Tech (ICT)')
```

9. Extract Faculty Contact / Email Information.

Find all contact links on the faculty cards (such as mailto: email links or phone numbers).

Extract the email addresses or telephone details and display them.

```
r = requests.get('https://darshan.ac.in/faculty-profile/Dr-Gopi-Bhupendrabhai-Sanghani')
s = BeautifulSoup(r.text, "html.parser")
data = s.find('span', class_='d-inline-block g-font-code')
email = s.find('span', class_='d-inline-block g-width-80x')

print(data.text.strip())
print(email.text.strip())

(0): 9727747317
gopi.sanghani@darshan.ac.in
```

10. Create a Pandas DataFrame of MCA Faculty Directory.

Collect all faculty Names, Designations, and Profile Links into separate lists.

Combine them into a structured pandas.DataFrame with columns ["Faculty Name", "Designation", "Profile URL"] and display the first 5 rows.

```
import pandas as pd
df = pd.DataFrame({
    "Faculty Name": names,
    "Designation": designations,
    "Profile URL": profiles_img
})
df
```

	Faculty Name	Designation	Profile URL	
0	Dr. Gopi Sanghani	Dean - Faculty of Computer Application	https://du-website.s3.ap-south-1.amazonaws.com...	
1	Prof. Swati Sharma	Assistant Professor	https://du-website.s3.ap-south-1.amazonaws.com...	
2	Prof. Maulik Trivedi	Assistant Professor	https://du-website.s3.ap-south-1.amazonaws.com...	
3	Prof. Firoz Sherasiya	Assistant Professor	https://du-website.s3.ap-south-1.amazonaws.com...	
4	Prof. Mayur Padia	Assistant Professor	https://du-website.s3.ap-south-1.amazonaws.com...	
5	Prof. Vaseem Ghada	Assistant Professor	https://du-website.s3.ap-south-1.amazonaws.com...	
6	Prof. Umesh Thoriya	Assistant Professor	https://du-website.s3.ap-south-1.amazonaws.com...	
7	Prof. Mehul Bhundiya	Assistant Professor	https://du-website.s3.ap-south-1.amazonaws.com...	
8	Prof. Jayesh Vagadiya	Assistant Professor	https://du-website.s3.ap-south-1.amazonaws.com...	
9	Prof. Rajkumar Gondaliya	Assistant Professor	https://du-website.s3.ap-south-1.amazonaws.com...	
10	Prof. Shruti Maniar	Assistant Professor	https://du-website.s3.ap-south-1.amazonaws.com...	
11	Prof. Dharmik Vasiyani	Assistant Professor	https://du-website.s3.ap-south-1.amazonaws.com...	
12	Prof. Devangi Kotak	Assistant Professor	https://du-website.s3.ap-south-1.amazonaws.com...	
13	Prof. Madhuresh Fichadiya	Assistant Professor	https://du-website.s3.ap-south-1.amazonaws.com...	

Next steps:

[Generate code with df](#)[New interactive sheet](#)



Python for Data Science (PDS)

Lab - 14

Roll No. : 122

Name : Yatharth Sanghavi

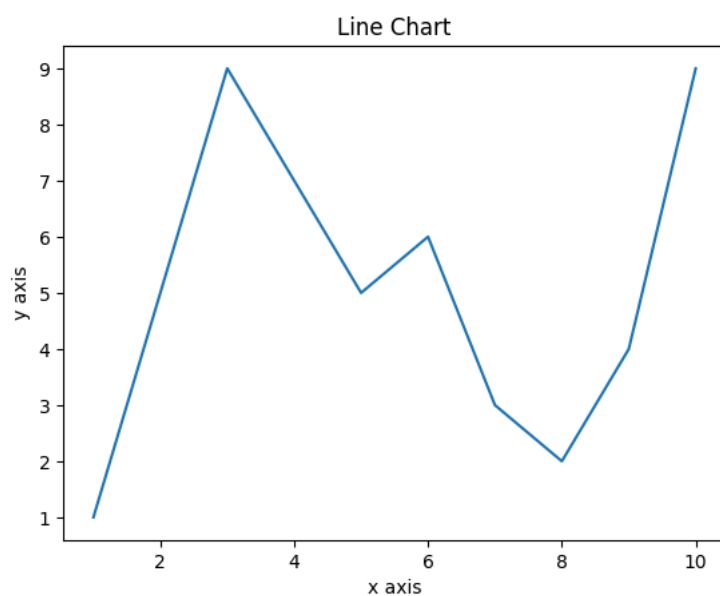
✓ Import matplotlib below

```
import matplotlib.pyplot as plt
```

✓ 01) WAP to display the line chart for given data for x and y axis.

```
x = range(1,11)  
y = [1,5,9,7,5,6,3,2,4,9]
```

```
plt.plot(x, y)  
plt.xlabel("x axis")  
plt.ylabel("y axis")  
plt.title("Line Chart")  
plt.show()
```

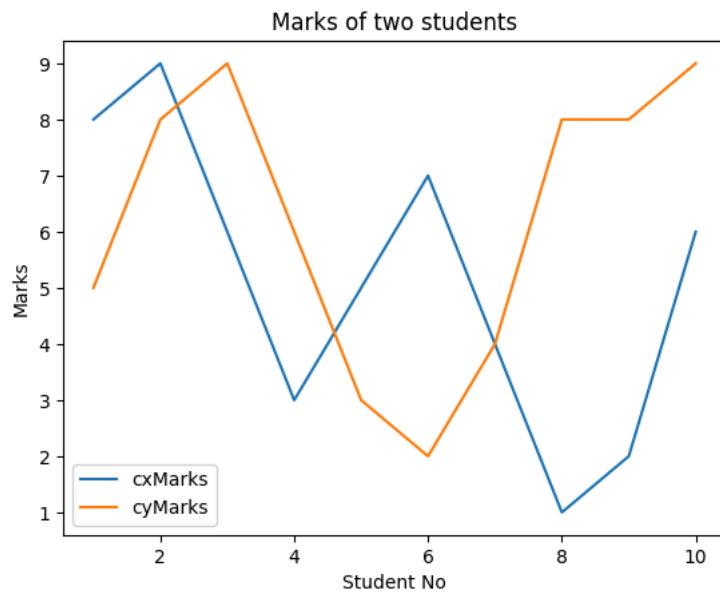


✓ 02) WAP to generate line chart for given data.

```
x = range(1,11,1)  
cxMarks= [8,9,6,3,5,7,4,1,2,6]
```

```
cyMarks= [5,8,9,6,3,2,4,8,8,9]
```

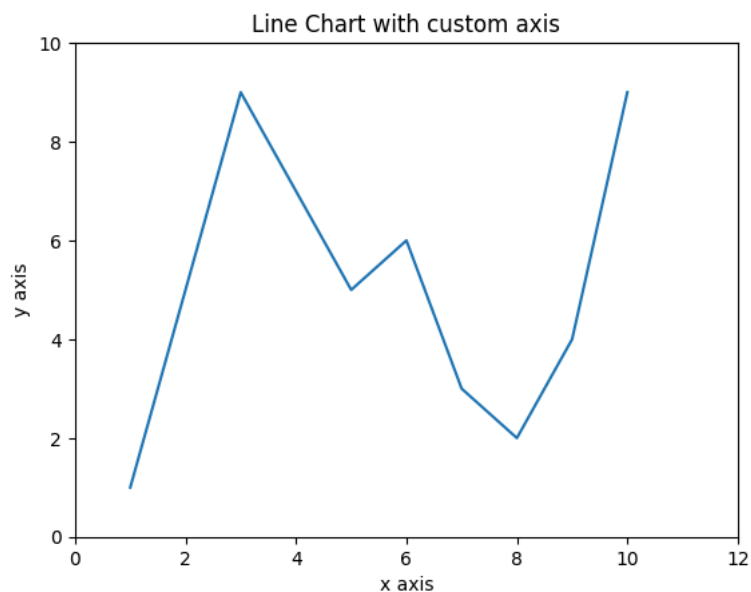
```
plt.plot(x, cxMarks, label="cxMarks")
plt.plot(x, cyMarks, label="cyMarks")
plt.xlabel("Student No")
plt.ylabel("Marks")
plt.title("Marks of two students")
plt.legend()
plt.show()
```



- 03) WAP to generate line chart for given data, but create your custom axis.

```
x = range(1,11)
y = [1,5,9,7,5,6,3,2,4,9]
```

```
plt.plot(x, y)
plt.axis([0, 12, 0, 10])
plt.xticks(range(0, 13, 2))
plt.yticks(range(0, 11, 2))
plt.xlabel("x axis")
plt.ylabel("y axis")
plt.title("Line Chart with custom axis")
plt.show()
```

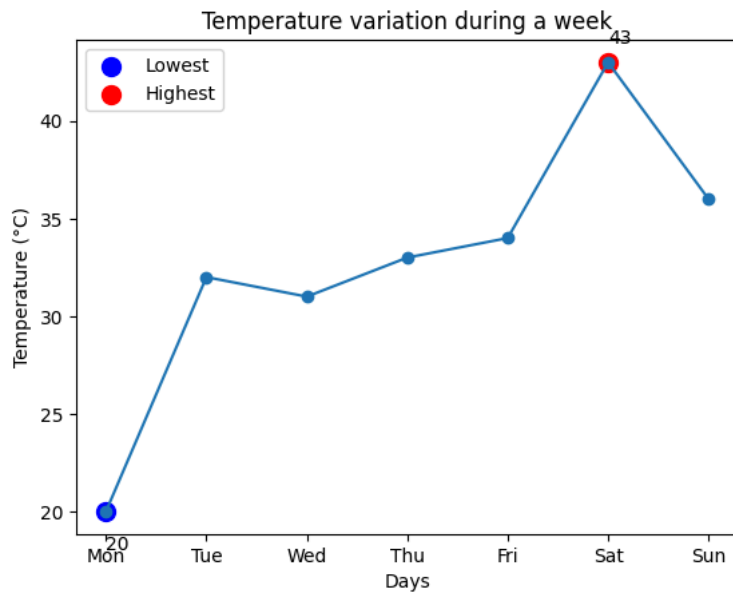


- 04) WAP to plot a line chart showing temperature variation during a week. Mark the lowest and highest temperature.

```
Days = ["Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"]
Temperature = [20, 32, 31, 33, 34, 43, 36]
```

```
min_temp = min(Temperature)
max_temp = max(Temperature)
min_day = Days[Temperature.index(min_temp)]
max_day = Days[Temperature.index(max_temp)]

plt.plot(Days, Temperature, marker="o")
plt.scatter(min_day, min_temp, color="blue", s=100, label="Lowest")
plt.scatter(max_day, max_temp, color="red", s=100, label="Highest")
plt.text(min_day, min_temp - 2, str(min_temp))
plt.text(max_day, max_temp + 1, str(max_temp))
plt.xlabel("Days")
plt.ylabel("Temperature (°C)")
plt.title("Temperature variation during a week")
plt.legend()
plt.show()
```



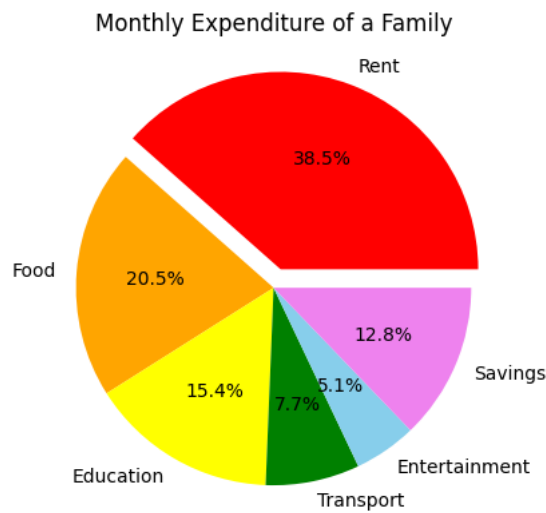
- ✓ 05) WAP to create a detailed pie chart representing the monthly expenditure distribution of a family.

The chart should display labels, percentage values, title, different colors for each slice, and explode one slice to highlight the highest expense.

```
Categories = ["Rent", "Food", "Education", "Transport", "Entertainment", "Savings"]
Amount = [15000, 8000, 6000, 3000, 2000, 5000]
```

```
colors = ["red", "orange", "yellow", "green", "skyblue", "violet"]
explode = [0.1, 0, 0, 0, 0, 0]

plt.pie(Amount, labels=Categories, colors=colors, explode=explode, autopct="%1.1f%%")
plt.title("Monthly Expenditure of a Family")
plt.show()
```

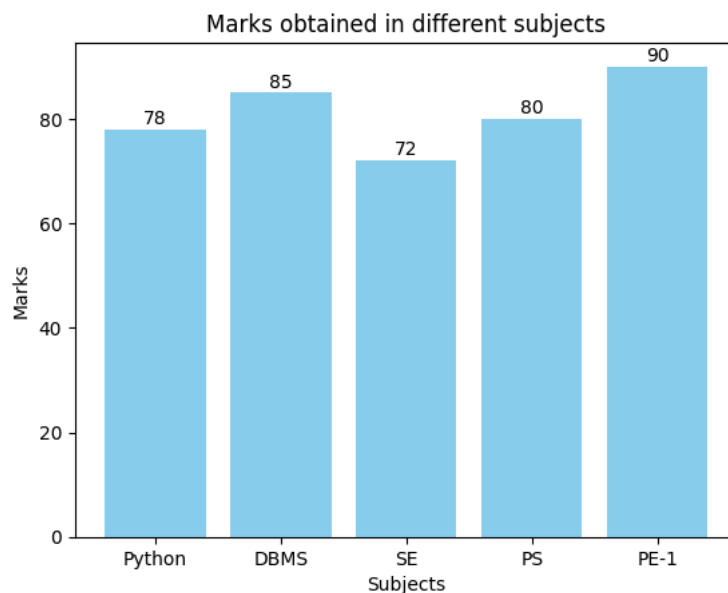


- ✓ 06) WAP to create bar chart showing the marks obtained by students in different subjects.

The program should also display the marks on the top of the respective bar.

```
Subjects = ["Python", "DBMS", "SE", "PS", "PE-1"]
Marks = [78, 85, 72, 80, 90]
```

```
plt.bar(Subjects, Marks, color="skyblue")
for i in range(len(Subjects)):
    plt.text(i, Marks[i] + 1, str(Marks[i]), ha="center")
plt.xlabel("Subjects")
plt.ylabel("Marks")
plt.title("Marks obtained in different subjects")
plt.show()
```



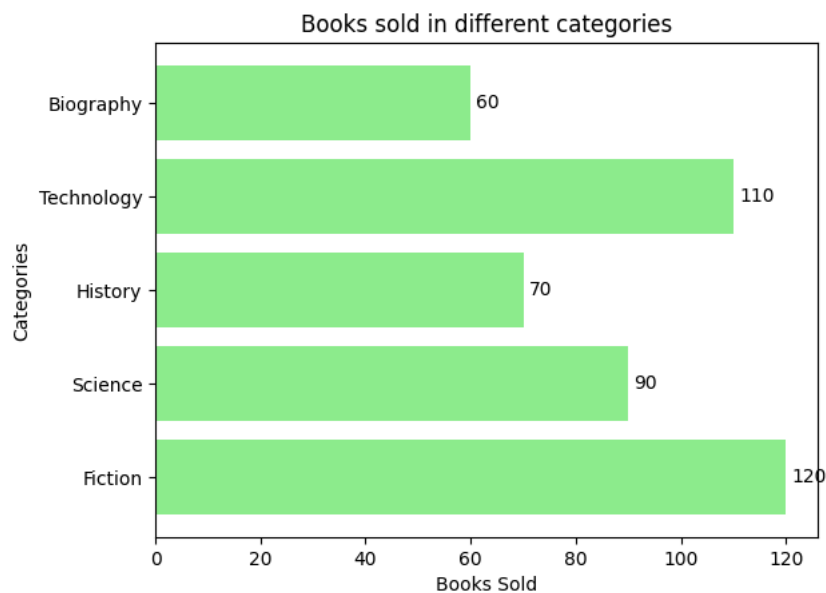
- ✓ 07) WAP to create a horizontal bar chart representing the number of books sold in different categories.

The program should also display the number of books sold at the end of the respective bar.

```
Categories = ["Fiction", "Science", "History", "Technology", "Biography"]
BooksSold = [120, 90, 70, 110, 60]
```

```
plt.barh(Categories, BooksSold, color="lightgreen")
for i in range(len(Categories)):
    plt.text(BooksSold[i] + 1, i, str(BooksSold[i]), va="center")
plt.xlabel("Books Sold")
plt.ylabel("Categories")
```

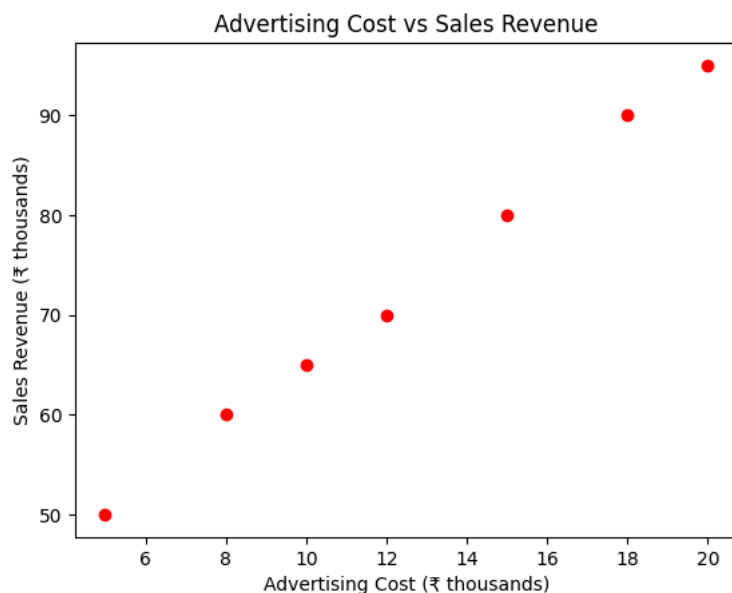
```
plt.title("Books sold in different categories")
plt.show()
```



08) WAP to create a scatter plot representing the relationship between advertising expenditure and sales revenue of a company.

```
AdvertisingCost = [5, 8, 10, 12, 15, 18, 20]
SalesRevenue = [50, 60, 65, 70, 80, 90, 95]
```

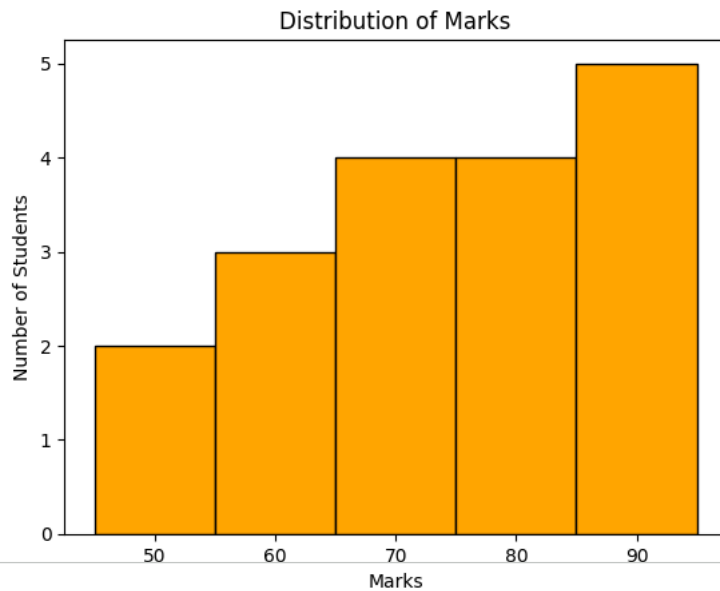
```
plt.scatter(AdvertisingCost, SalesRevenue, color="red")
plt.xlabel("Advertising Cost (₹ thousands)")
plt.ylabel("Sales Revenue (₹ thousands)")
plt.title("Advertising Cost vs Sales Revenue")
plt.show()
```



09) WAP to create a histogram representing the distribution of marks obtained by students in a class.

```
Marks = [45, 50, 55, 60, 62, 65, 68, 70, 72, 75, 78, 80, 82, 85, 88, 90, 92, 95]
```

```
plt.hist(Marks, bins=5, edgecolor="black", color="orange")
plt.xlabel("Marks")
plt.ylabel("Number of Students")
plt.title("Distribution of Marks")
plt.show()
```

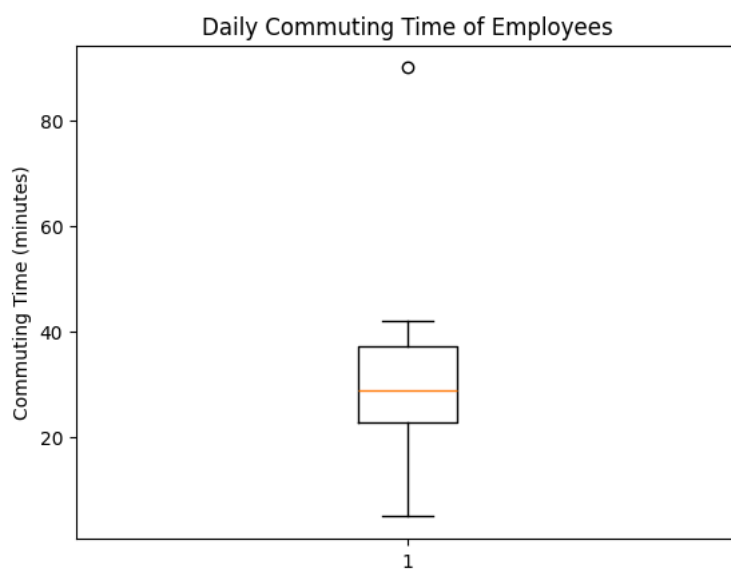


- 10) WAP to create a box plot representing the distribution of daily commuting time (in minutes) of employees and identify outliers (if any).

```
CommutingTime = [5, 18, 20, 22, 25, 27, 28, 30, 32, 35, 38, 40, 42, 90]
```

```
plt.boxplot(CommutingTime)
plt.ylabel("Commuting Time (minutes)")
plt.title("Daily Commuting Time of Employees")
plt.show()

# Outlier check using the 1.5 * IQR rule
data = sorted(CommutingTime)
n = len(data)
q1 = data[n // 4]
q3 = data[(3 * n) // 4]
iqr = q3 - q1
lower = q1 - 1.5 * iqr
upper = q3 + 1.5 * iqr
outliers = [v for v in data if v < lower or v > upper]
print("Outliers:", outliers)
```



```
Outliers: [90]
```



Python for Data Science - 2305CS303

Lab - 15

Roll No. : 122

Name : Yatharth Sanghavi

✓ Given an image file:

```
from PIL import Image
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

✓ 01) Read the image using the PIL library.

```
img1 = Image.open("download.jfif")
```

```
plt.imshow(img1)  
plt.show()
```



```
plt.imshow(img1)  
plt.axis("off")  
plt.show()
```



Darshan

UNIVERSITY

- ✓ 02) Convert the image into a NumPy array.

```
img1_array = np.array(img1)
```

```
print(img1_array.shape)
```

```
(675, 1705, 3)
```

- ✓ 03) Create a copy of the image array.

```
img_copy = img1_array.copy()
```

- ✓ 04) Extract a sub-image starting from the 100th row and 100th column, selecting every 20th row and column.

```
subimg = img_copy[100::20, 100::20]
```

- ✓ 05) Display the processed image using Matplotlib.

```
plt.imshow(subimg)  
plt.axis("off")  
plt.show()
```



Darshan

UNIVERSITY